

Numerical Analysis Midterm Project

B11501015 廖致翔

Q1: Consider the function $y(x) = \cos(10x^2)\exp(-x)$

#誤差定義

$$e_n(x) = p_{n-1}(x) - f(x).$$

使用兩個整體誤差指標： $E_\infty = \max_{x \in [-1,1]} |e_n(x)|$, $E_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{m=1}^N e_n(x_m)^2}$.

其中 E_∞ 反映最壞情況誤差， E_{RMS} 則反映全區間平均意義下的誤差量級。

(a) Implement the polynomial interpolation to interpolate the function value with points that are evenly spaced in the range $-1 \leq x \leq 1$. Investigate the error distribution for several numbers of interpolating points.

(i) 在區間 $[-1,1]$ 上取 n 個等距節點 $x_i = -1 + \frac{2(i-1)}{n-1}$, $i = 1, 2, \dots, n$, 並令

$f(x) = \cos(10x^2)e^{-x}$. 對每一組節點，使用 **Newton divided differences** 建立 $n-1$ 次插值多項式 $p_{n-1}(x)$ ，並於 $[-1,1]$ 上取 5001 個等距細網格點評估誤差

$$e_n(x) = p_{n-1}(x) - f(x).$$

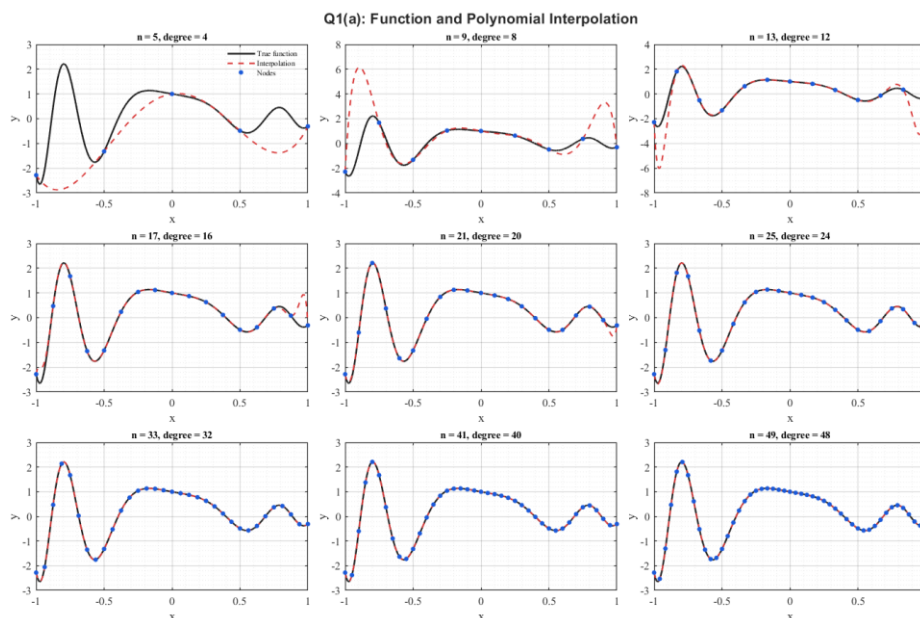
比較的插值點數為 $n = 5, 9, 13, 17, 21, 25, 33, 41, 49$.

(ii) MATLAB code 見 Appendix

Table 1: Q1(a) 在細網格上的誤差統計結果

插值點數 n	多項式次數 $n-1$	最大絕對誤差 E_∞	均方根誤差 E_{RMS}	最大誤差位置
5	4	5.0438	1.5326	$x \approx -0.7996$
9	8	7.2995	1.8605	$x \approx -0.9264$
13	12	3.4480	0.82447	$x \approx -0.9592$
17	16	1.3201	0.22771	$x \approx 0.9676$
21	20	0.37797	0.053481	$x \approx 0.9756$
25	24	0.075175	0.010775	$x \approx 0.9804$
33	32	0.0013513	0.00019648	$x \approx -0.9856$
41	40	1.1331×10^{-5}	1.3373×10^{-6}	$x \approx -0.9892$
49	48	3.0011×10^{-5}	2.5288×10^{-6}	$x \approx 0.9876$

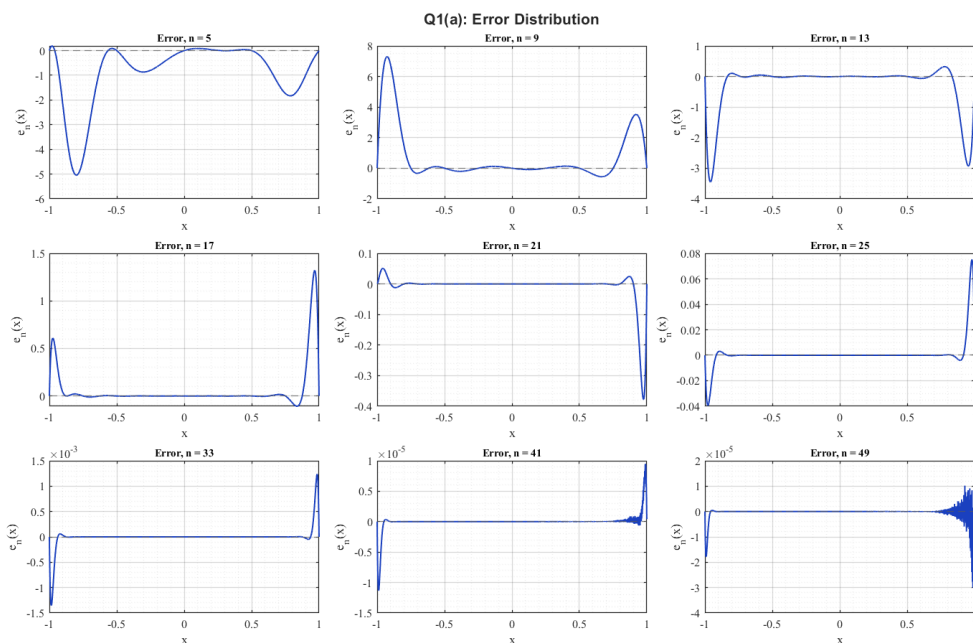
可看出三個現象。第一，當插值點數由 $n=13$ 起持續增加時，誤差整體快速下降，顯示高次多項式在這類平滑函數上確實能提供高精度逼近。第二，最大誤差並非一開始就單調下降；從 $n=5$ 增加到 $n=9$ 時，最大誤差反而變大。第三，當 n 增加到很大時，誤差主要集中在端點附近，而且在 $n=41$ 與 $n=49$ 之間，最大誤差又有些微回升，顯示等距節點高次插值在端點附近仍具有敏感性。



▲原函數與插值函數比較

黑線為原始函數，紅色虛線為插值多項式

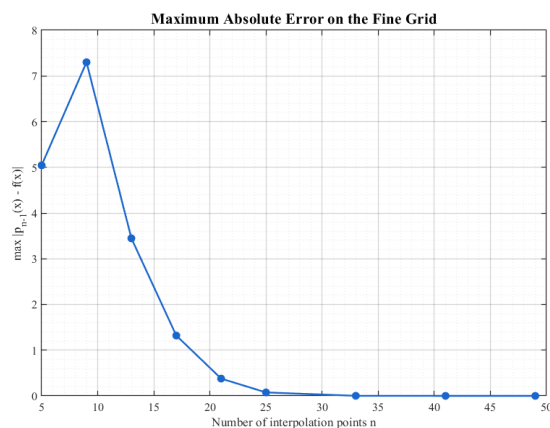
當 $n=5$ 與 $n=9$ 時，插值曲線與原函數仍有明顯差距，特別是在區間兩端，局部振盪較為顯著，其中 $n=9$ 甚至比 $n=5$ 更差。當 n 增加到 13,17,21 後，插值曲線已明顯貼近原函數；到了 $n=25,33,41,49$ 時，除了端點附近外，兩者幾乎重合。這表示增加節點數確實能改善整體逼近效果，但這種改善先發生在區間中央，而非在整個區間上均勻同步發生。



▲誤差分布圖

由函數比較圖與誤差分布圖可見，當 $n=5$ 與 $n=9$ 時，插值在整個區間內仍有明顯偏差，且 $n=9$ 的表現甚至較差；隨著 n 增加，中間區域的誤差迅速下降，而主要誤差逐漸集中於端點附近。當 $n=33,41,49$ 時，區間中央的誤差已極小，最

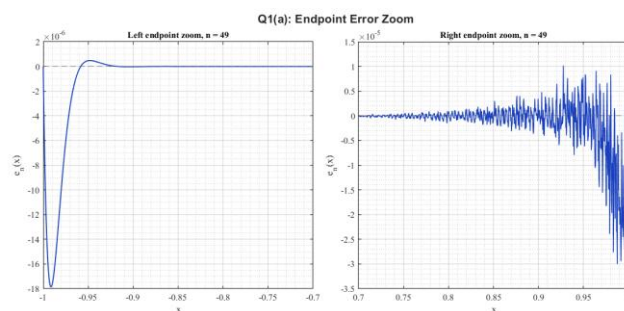
大誤差幾乎完全由端點控制，顯示等距高次多項式插值具有明顯的端點敏感性。



▲最大誤差趨勢圖

此圖顯示最大絕對誤差 E_∞ 隨插值點數 n 的變化趨勢。從圖中可見，最大誤差在初期並未單調下降；當 $n=9$ 時反而大於 $n=5$ 。之後隨著點數增加，誤差快速下降，到 $n=41$ 時已降至約 1.1331×10^{-5} 。然而，當 n 再增加到 49 時，最大誤差又略為回升到 3.0011×10^{-5} 。

這說明對等距節點而言，高次全域多項式插值雖然可以達到非常高的精度，但其最壞情況誤差並不保證絕對單調收斂，端點附近的局部振盪仍可能影響整體表現。



▲端點局部放大圖

此圖為 $n=49$ 時端點附近的誤差局部放大圖。由左圖可見，左端點附近存在一個明顯的負向尖峰；右圖則顯示右端點附近有更強的局部振盪，且振幅在接近 $x=1$ 時快速放大。這說明當節點數很高時，整體誤差雖已很小，但端點仍然是最脆弱的區域，而且右端點的局部不穩定性更為明顯。

(b) Interpolate the function value again but this time with the Chebyshev points in the range $-1 \leq x \leq 1$. Investigate the error distribution for several numbers of interpolating points.

(i) Chebyshev 節點:

若共有 n 個插值點，則在區間 $[-1,1]$ 上取 $x_j = -\cos\left(\frac{(2j-1)\pi}{2n}\right)$, $j=1,2,\dots,n$.

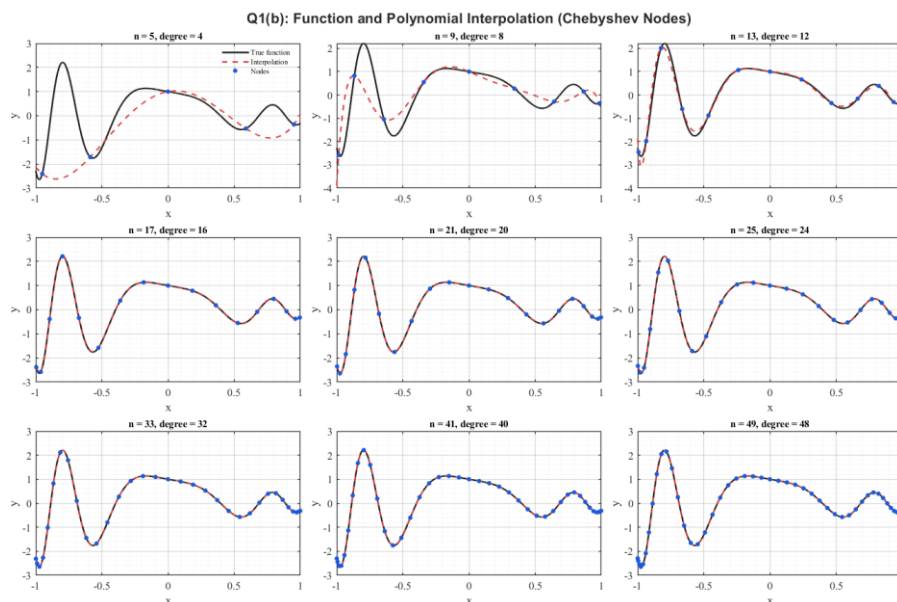
這組節點在兩端較密、中間較疏，可有效改善等距節點高次插值在端點附近的振盪問題。

(ii) MATLAB code 見 Appendix

Table 1: Q1(b) 在細網格上的誤差統計結果

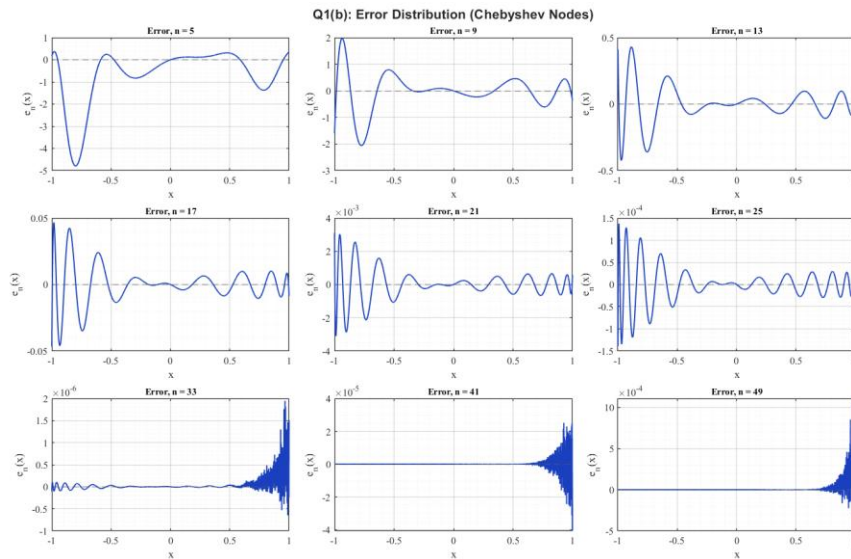
插值點數 n	多項式次數 $n-1$	最大絕對誤差 E_{∞}	均方根誤差 E_{RMS}	最大誤差位置
5	4	4.7911	1.3953	$x \approx -0.7996$
9	8	2.0612	0.65992	$x \approx -0.7740$
13	12	0.42973	0.13092	$x \approx -0.8860$
17	16	0.046733	0.013508	$x \approx -1$
21	20	0.0031192	0.00085854	$x \approx -1$
25	24	0.00013858	3.7078×10^{-5}	$x \approx -1$
33	32	1.9481×10^{-6}	1.5345×10^{-7}	$x \approx 0.9656$
41	40	4.0626×10^{-5}	2.6904×10^{-6}	$x \approx 1$
49	48	0.0011068	6.9601×10^{-5}	$x \approx 0.9952$

由表可見，當 n 由 5 增加到 33 時，最大誤差與均方根誤差皆快速下降，代表 Chebyshev 節點對函數具有明顯的誤差抑制效果。其中 $n=33$ 時，最大絕對誤差已降至約 1.95×10^{-6} 。然而，當 n 再增加到 41 與 49 時，最大誤差反而回升，顯示在非常高次時，數值方法本身的浮點誤差與係數敏感性開始影響結果。



▲原函數與插值函數比較

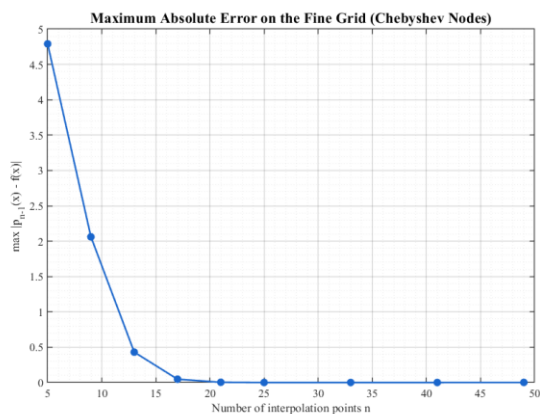
從函數比較圖可看出，當 $n=5$ 時，插值曲線仍無法充分掌握原函數的整體波形；但從 $n=9$ 開始，插值結果已明顯優於等距節點的情況。當 $n=13$ 、17、21 之後，插值曲線已高度貼近真實函數；到了 $n=25$ 與 $n=33$ 時，兩者幾乎重合。這表示 Chebyshev 節點在相同節點數下，通常能提供更穩定的逼近效果，尤其在端點附近的表現明顯優於等距節點。



▲誤差分布圖

誤差分布圖顯示，當 $n=5$ 與 $n=9$ 時，誤差仍遍布整個區間，但其端點振盪已比等距節點小得多。當 $n=13$ 、 17 、 21 、 25 時，誤差快速縮小，而且分布相對平順，說明 Chebyshev 節點確實能有效壓低端點附近的誤差放大。

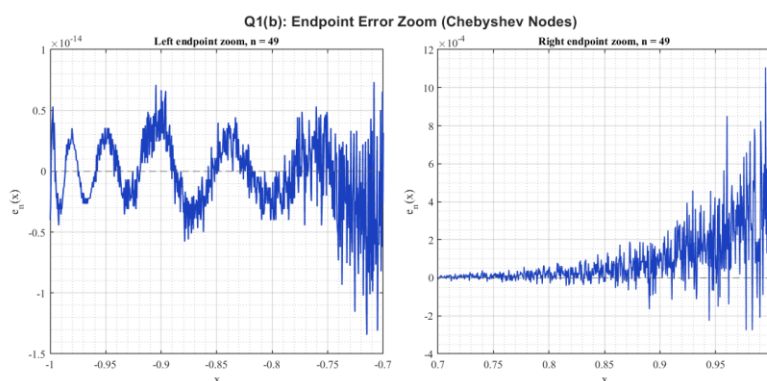
然而，在 $n=33$ 、 41 、 49 的情況下，誤差開始逐漸集中到右端點附近，尤其 $n=49$ 時右端點附近出現明顯振盪，並成為最大誤差的主要來源。這表示當插值次數非常高時，誤差行為已不再只由理論插值誤差控制，而開始受到數值運算精度影響。



▲最大誤差趨勢圖

最大絕對誤差趨勢圖顯示，從 $n=5$ 到 $n=33$ ，誤差快速下降，符合使用 Chebyshev 節點後端點誤差被有效控制的理論預期。這也說明相較於等距節點，Chebyshev 節點能更快得到高精度結果。

不過，當 n 增加到 41 與 49 時，最大誤差不再持續下降，反而分別回升。因此，本題結果顯示：Chebyshev 節點雖能大幅改善插值表現，但若仍直接以高次 Newton divided differences 計算，當次數過高時，數值穩定性仍可能成為限制。



▲端點局部放大圖

端點局部放大圖顯示，當 $n=49$ 時，左端點附近的誤差量級約在 10^{-14} ；相對地，右端點附近的誤差則明顯較大，且在靠近 $x=1$ 時快速放大至約 10^{-4} 到 10^{-3} 量級。這說明在本題中，Chebyshev 節點已成功壓低左端點誤差，但右端點仍出現局部振盪。整體最大誤差回升主要來自右端點附近的局部數值不穩定，而非整個區間的普遍惡化。

(iii) 與 Q1(a) 的比較

若與 Q1(a) 的等距節點結果相比，Chebyshev 節點在大多數中等點數下均有明顯優勢。例如在 $n=25$ 時，Q1(a) 的最大誤差約為 7.52×10^{-2} ，而 Q1(b) 已降至 1.39×10^{-4} ；在 $n=33$ 時，Q1(a) 約為 1.35×10^{-3} ，Q1(b) 則已降至 1.95×10^{-6} 。這說明 Chebyshev 節點確實能有效抑制端點附近的誤差放大現象。

不過，在 $n=41$ 與 $n=49$ 的結果也說明：節點分布雖然重要，但數值實作方式同樣關鍵。若多項式次數過高，Newton divided differences 在有限精度下仍可能導致誤差回升。

(c) Implement the cubic spline interpolation to interpolate the function value with the grid points in (1a). Investigate the error distribution for several numbers of interpolating points.

(i) 等距節點，設 $a = x_1 < x_2 < \dots < x_n = b$ ， $a = -1$ ， $b = 1$ ，且節點為等距分布。

令 $f_j = f(x_j)$ ， $j = 1, 2, \dots, n$ ，並記 $\Delta_j = x_{j+1} - x_j$ ， $j = 1, 2, \dots, n-1$ 。

(ii) cubic spline interpolation

設 $S_4 f(x)$ 為通過所有節點的 cubic spline。樣條函數在每一個區間 $[x_j, x_{j+1}]$ 上都是三次多項式，並且滿足函數值、一階導數與二階導數在各節點處皆連續。

可令 $(S_4 f(x))'' = S_2 f''(x)$ ，亦即先對 $f''(x)$ 作分段線性插值，再積分兩次得到

$S_4 f(x)$ 。若記 $f_j'' \equiv (S_4 f)''(x_j)$ ，則在區間 $[x_j, x_{j+1}]$ 上，cubic spline 可寫為

$$S_4 f(x) = \left[\frac{(x_{j+1} - x)^3}{6\Delta_j} - \frac{\Delta_j(x_{j+1} - x)}{6} \right] f_j'' + \left[\frac{(x - x_j)^3}{6\Delta_j} - \frac{\Delta_j(x - x_j)}{6} \right] f_{j+1}'' + f[x_j, x_{j+1}](x - x_j) + f_j.$$

(iii) Natural spline 條件

邊界條件為 $f_1'' = f_n'' = 0$. 對內部節點 $j = 2, 3, \dots, n-1$ ，二階導數滿足三對角線性

$$\text{系統 } \frac{\Delta_{j-1}}{6} f_{j-1}'' + \frac{\Delta_{j-1} + \Delta_j}{3} f_j'' + \frac{\Delta_j}{6} f_{j+1}'' = f[x_j, x_{j+1}] - f[x_{j-1}, x_j].$$

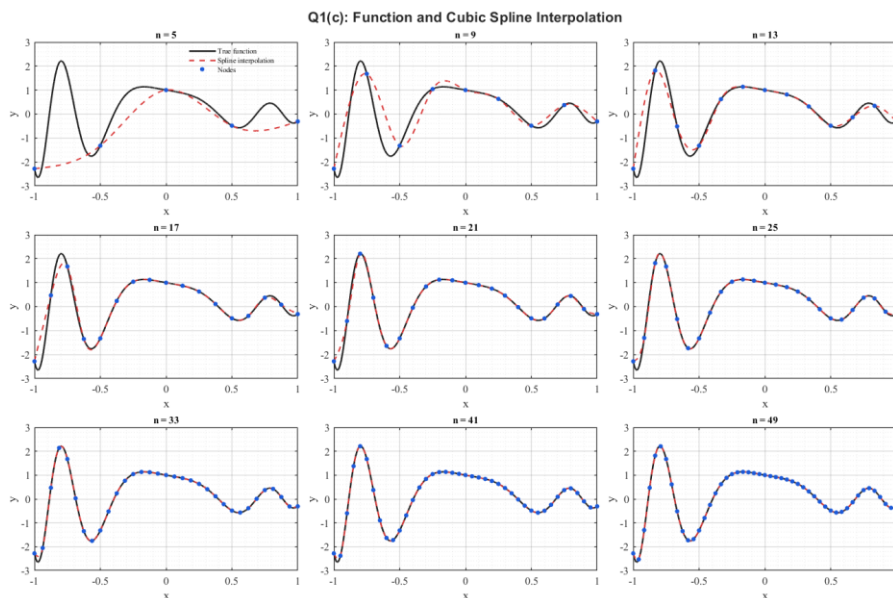
解出所有 f_j'' 後，即可在各子區間上組成完整的三次樣條函數。

(iv) MATLAB code 見 Appendix

Table 1: Q1(c) 在細網格上的誤差統計結果

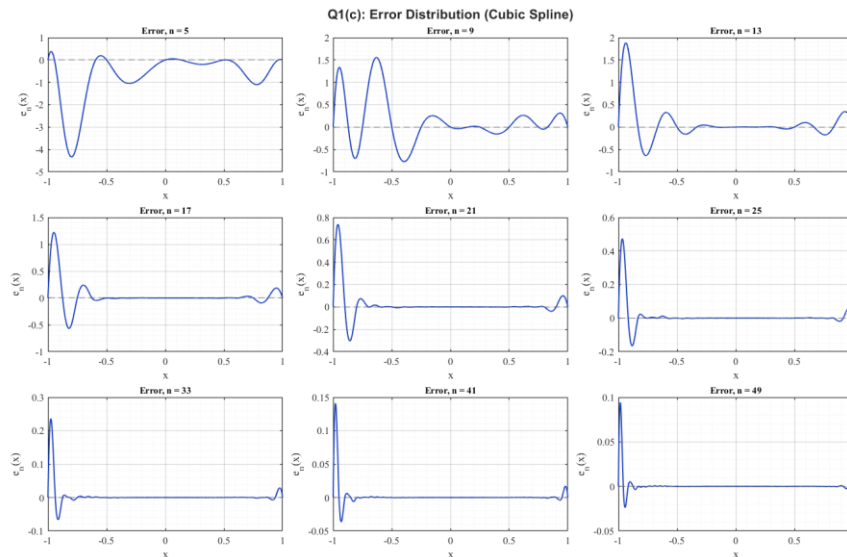
節點數 n	多項式次數 $n-1$	最大絕對誤差 E_∞	均方根誤差 E_{RMS}	最大誤差位置
5	4	4.3315	1.2561	$x \approx -0.7988$
9	8	1.5566	0.51717	$x \approx -0.6312$
13	12	1.8816	0.41077	$x \approx -0.9372$
17	16	1.2228	0.24325	$x \approx -0.9492$
21	20	0.73826	0.12692	$x \approx -0.9592$
25	24	0.47288	0.07227	$x \approx -0.9664$
33	32	0.23599	0.030517	$x \approx -0.9752$
41	40	0.14101	0.016204	$x \approx -0.9804$
49	48	0.094033	0.0098485	$x \approx -0.9840$

由表可見，當節點數增加時，誤差整體呈下降趨勢，但其下降速度明顯比 Q1(b) 的 Chebyshev 多項式插值慢。特別是從 $n=13$ 之後， E_∞ 與 E_{RMS} 皆逐步下降，說明 cubic spline 對函數具有穩定的收斂表現。不過，從 $n=9$ 到 $n=13$ 時，最大誤差有一次小幅上升，顯示誤差並非完全單調下降。



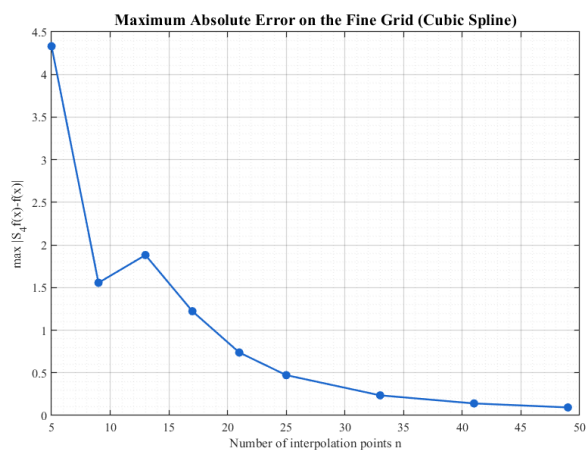
▲原函數與插值函數比較

由函數比較圖可看出，當 $n=5$ 時，樣條函數與原函數仍存在明顯差距，尤其在左端附近與局部峰值區域偏差較大。當節點數增加到 9、13、17 後，樣條曲線已逐漸貼近原函數；到 $n=25$ 、33、41、49 時，除端點附近外，兩者幾乎重合。這說明 **cubic spline** 作為局部插值方法，能在不使用高次全域多項式的情況下，穩定重現函數的整體波形。相較於 Q1(a) 的等距高次多項式插值，這種方法沒有明顯的全區間振盪問題。



▲誤差分布圖

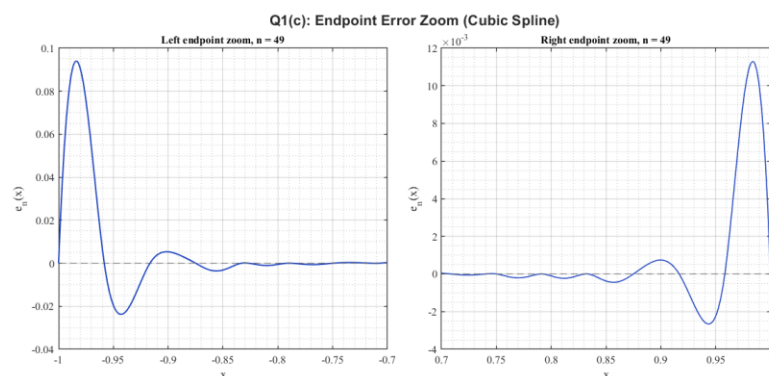
由誤差分布圖可見，當 $n=5$ 時，誤差在整個區間內都偏大；當節點數增加後，區間中央的誤差迅速下降，而主要誤差逐漸集中到左端附近。從 $n=17$ 起，中間區域的誤差已相當小，主要誤差幾乎完全由左端點附近控制。這表示對本題而言，**cubic spline** 的主要誤差來源不是整體逼近不足，而是邊界附近的局部誤差。與等距全域多項式不同，這裡看不到明顯的 **Runge** 現象；誤差更像是局部邊界效應，而非整段區間的劇烈振盪。



▲最大誤差趨勢圖

最大絕對誤差趨勢圖顯示，從 $n=5$ 到 $n=49$ ，誤差整體持續下降。雖然在 $n=13$ 時相較於 $n=9$ 有一個小幅回升，但之後下降趨勢相當穩定，並沒有像 Q1(a) 或 Q1(b) 高次多項式那樣在高階時出現明顯反彈。

這反映了 cubic spline 的一個優點：由於它是分段三次多項式，並非以單一高次多項式逼近整個區間，因此整體數值穩定性較好。從結果來看，cubic spline 的精度不如 Q1(b) 的 Chebyshev 多項式那樣高，但其表現較為平穩。



▲端點局部放大圖

端點局部放大圖顯示，當 $n=49$ 時，左端點附近的誤差量級約為 10^{-1} ，而右端點附近約為 10^{-3} 。這表示本題的最大誤差主要來自左端點，而不是右端點。結合上表可見，最大誤差位置始終落在靠近左端的位置，且隨著節點數增加逐漸逼近 $x = -1$ 。

造成此現象的主要原因，是本題採用 natural spline 邊界條件 $f_1'' = f_n'' = 0$ 。當真實函數在邊界附近的曲率與 natural spline 條件不完全相符時，邊界區域便容易成為誤差最集中的地方。因此，Q1(c) 的誤差主要來自邊界條件所引入的局部偏差，而非全域高次插值的不穩定性。

(vi) 與 Q1(a) 及 Q1(b) 的比較

若與 Q1(a) 的等距高次多項式相比，cubic spline 顯然較穩定。Q1(a) 在高次時的誤差會強烈集中於端點，且曾出現明顯的端點振盪；相較之下，Q1(c) 雖然誤差仍由端點主導，但其下降趨勢更平穩，也沒有嚴重的全域振盪。

若與 Q1(b) 的 Chebyshev 多項式相比，則 Q1(c) 的精度明顯較低。例如在 $n=49$ 時，Q1(b) 的最大誤差量級已可達到約 10^{-3} 以下甚至更低，而 Q1(c) 仍維持在約 10^{-1} 。這表示對本題這種平滑函數而言，Chebyshev 節點的高次多項式插值在精度上仍優於 cubic spline。不過，cubic spline 的優勢在於局部性與穩定性，較不容易出現高次全域多項式的端點劇烈振盪。

Q2: Consider the (continuous) functions

$$g(x; a) = e^{-ax^2} \quad \text{and} \quad f(x; a) = \sum_{m=-\infty}^{\infty} g(x - 2\pi m; a). \quad (1)$$

(a) Find the continuous Fourier transform of $g(x; a)$ on $x \in (-\infty, \infty)$, $G(s; a)$ on $s \in (-\infty, \infty)$, and analytical expression for the Fourier series coefficients of $f(x; a)$ on $x \in [-\pi, \pi]$, $\hat{f}_j(a)$ on $j \in \mathbb{I}(-\infty, \infty)$. These will be helpful for the next part. What is the mathematical relation between $G(s; a)$ and $\hat{f}_j(a)$?

由定義可知， $f(x; a)$ 為 $g(x; a)$ 以週期 2π 作 periodic replication 後得到的週期函數，因此關鍵在於建立 non-periodic 的 continuous Fourier transform 與 periodic 的 Fourier series coefficients 之間的對應關係。

$g(x; a)$ 的 continuous Fourier transform 定義為

$$G(s; a) = \int_{-\infty}^{\infty} g(x; a) e^{-i2\pi s x} dx = \int_{-\infty}^{\infty} e^{-ax^2} e^{-i2\pi s x} dx.$$

將指數項配方： $-ax^2 - i2\pi s x = -a \left(x + \frac{i\pi s}{a} \right)^2 - \frac{\pi^2 s^2}{a}$,

因此
$$G(s; a) = e^{-\pi^2 s^2 / a} \int_{-\infty}^{\infty} e^{-a \left(x + \frac{i\pi s}{a} \right)^2} dx = \sqrt{\frac{\pi}{a}} e^{-\pi^2 s^2 / a}.$$

接著 $f(x; a)$ 在 $x \in [-\pi, \pi]$ 上的 Fourier series coefficients：

$$\hat{f}_j(a) = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x; a) e^{-ijx} dx, \quad j \in \mathbb{Z}.$$

將 $f(x; a)$ 代入後， $\hat{f}_j(a) = \frac{1}{2\pi} \int_{-\pi}^{\pi} \left(\sum_{m=-\infty}^{\infty} g(x - 2\pi m; a) \right) e^{-ijx} dx$.

由於 $e^{-ij2\pi m} = 1$ ($j, m \in \mathbb{Z}$)，再令 $y = x - 2\pi m$ ，可得

$$\hat{f}_j(a) = \frac{1}{2\pi} \sum_{m=-\infty}^{\infty} \int_{-\pi-2\pi m}^{\pi-2\pi m} g(y; a) e^{-ijy} dy = \frac{1}{2\pi} \int_{-\infty}^{\infty} g(y; a) e^{-ijy} dy.$$

(各區間 $[-\pi - 2\pi m, \pi - 2\pi m]$ ($m \in \mathbb{Z}$) 恰好無重疊地覆蓋整條實數軸)

而 continuous Fourier transform 的 kernel 為 $e^{-i2\pi s y}$ ，故只要令

$$2\pi s = j \Rightarrow s = \frac{j}{2\pi}, \text{ 便有 } \hat{f}_j(a) = \frac{1}{2\pi} G\left(\frac{j}{2\pi}; a\right).$$

代入 $G(s; a)$ 的解析式，即得

$$\hat{f}_j(a) = \frac{1}{2\pi} \sqrt{\frac{\pi}{a}} \exp\left(-\frac{\pi^2}{a} \left(\frac{j}{2\pi}\right)^2\right) = \frac{1}{2\sqrt{\pi a}} e^{-j^2/(4a)}.$$

因此，最重要的數學關係為
$$\hat{f}_j(a) = \frac{1}{2\pi} G\left(\frac{j}{2\pi}; a\right)$$

亦即， $f(x; a)$ 的 Fourier series coefficients，正是 $g(x; a)$ 的 continuous Fourier transform 在離散頻率 $s = j/(2\pi)$ 上的取樣，再乘上一個 $1/(2\pi)$ 的尺度因子。

(b) Write a program to use the FFT to evaluate the DFT coefficients of the function $g(x; a)$ on $x \in [-\pi, \pi]$ with $N + 1$ points, where N is a power of 2. Let's called these DFT coefficients, $\tilde{g}_j(a, N)$ on $j \in \mathbb{I}(-\frac{N}{2}, \frac{N}{2} - 1)$.

先取等距點，由於 $x_0 = -\pi$ 與 $x_N = \pi$ 在週期下代表同一位置，因此 FFT 實際只使用前 N 個互異點

$$x_k = -\pi + \frac{2\pi k}{N}, \quad k = 0, 1, \dots, N-1.$$

依題目定義，

$$\tilde{g}_j(a, N) = \frac{1}{N} \sum_{k=0}^{N-1} g(x_k; a) e^{-ijx_k}. \text{ 將 } x_k = -\pi + \frac{2\pi k}{N} \text{ 代入可得}$$

$$\tilde{g}_j(a, N) = \frac{1}{N} \sum_{k=0}^{N-1} g(x_k; a) e^{-ij\left(-\pi + \frac{2\pi k}{N}\right)} = e^{ij\pi} \frac{1}{N} \sum_{k=0}^{N-1} g(x_k; a) e^{-i2\pi jk/N}.$$

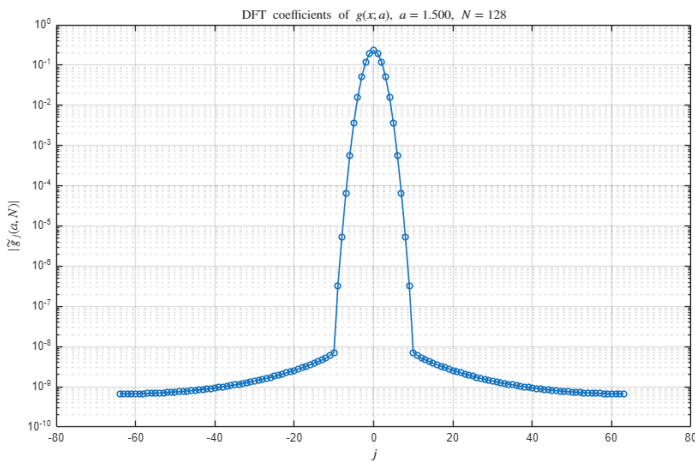
因此若令 $G_j^{\text{FFT}} = \frac{1}{N} \sum_{k=0}^{N-1} g(x_k; a) e^{-i2\pi jk/N}$, 則 $\tilde{g}_j(a, N) = e^{ij\pi} G_j^{\text{FFT}}$

亦即，題目要求之 DFT coefficients 與 MATLAB 直接輸出的 FFT 結果，只差一個由區間 $[0, 2\pi]$ 平移至 $[-\pi, \pi]$ 所產生的 phase factor $e^{ij\pi} = (-1)^j$ 。實作時，先以 `fft` 計算頻譜，再以 `fftshift` 將索引重排成 $j = -\frac{N}{2}, -\frac{N}{2} + 1, \dots, \frac{N}{2} - 1$,

最後乘上 $e^{ij\pi}$ 即得 $\tilde{g}_j(a, N)$ 。

為驗證程式正確性，另以定義式直接計算

$\tilde{g}_j(a, N) = \frac{1}{N} \sum_{k=0}^{N-1} g(x_k; a) e^{-ijx_k}$, 並與 FFT 結果比較。取 $a = 1.5, N = 128$ 作測試，得到 MATLAB 結果如下：



j	real_gtilde_fft	imag_gtilde_fft	abs_gtilde_fft	abs_gtilde_direct	abs_error
-8	5.3612e-06	2.048e-19	5.3612e-06	5.3612e-06	4.2614e-17
-7	6.5413e-05	5.2854e-18	6.5413e-05	6.5413e-05	2.6528e-17
-6	0.00057092	-6.486e-19	0.00057092	0.00057092	2.83e-17
-5	0.003571	-1.3574e-18	0.003571	0.003571	1.3082e-18
-4	0.016004	5.1834e-18	0.016004	0.016004	9.6057e-18
-3	0.051393	1.5521e-17	0.051393	0.051393	2.5118e-17
-2	0.11826	3.0482e-17	0.11826	0.11826	2.5538e-17
-1	0.19497	2.4094e-17	0.19497	0.19497	3.5544e-17
0	0.23033	0	0.23033	0.23033	2.7756e-17
1	0.19497	-2.4094e-17	0.19497	0.19497	3.5544e-17
2	0.11826	-3.0482e-17	0.11826	0.11826	2.5538e-17
3	0.051393	-1.5521e-17	0.051393	0.051393	2.5118e-17
4	0.016004	-5.1834e-18	0.016004	0.016004	9.6057e-18
5	0.003571	1.3574e-18	0.003571	0.003571	1.3082e-18
6	0.00057092	6.486e-19	0.00057092	0.00057092	2.83e-17
7	6.5413e-05	-5.2854e-18	6.5413e-05	6.5413e-05	2.6528e-17
8	5.3612e-06	-2.048e-19	5.3612e-06	5.3612e-06	4.2614e-17

a = 1.500000
N = 128
max |gtilde_fft - gtilde_direct| = 6.377178e-16
max imaginary part of gtilde_fft = 3.048202e-17

(MATLAB code 見 Appendix)

$\max_j |\tilde{g}_j^{\text{FFT}} - \tilde{g}_j^{\text{direct}}| = 6.377178 \times 10^{-16}$, 且 $\max_j |\Im(\tilde{g}_j^{\text{FFT}})| = 3.048202 \times 10^{-17}$. 可見 FFT

與 direct summation 的差異已很小，表示 FFT 實作、索引重排與 phase correction 皆正確。

另外，由於 $g(x;a) = e^{-ax^2}$ 為實偶函數，其 DFT coefficients 應近似為實數且滿足

$\tilde{g}_{-j}(a,N) \approx \tilde{g}_j(a,N)$. 數值結果確實符合此性質。頻譜亦呈現以 $j=0$ 為中心、隨 $|j|$ 增加快速衰減的型態，表示 Gaussian 函數的能量主要集中於低波數區域。

(c) We want to isolate and evaluate three potential errors: truncation of the Fourier coefficients, sampling/aliasing, and non-periodicity/leakage. Discuss how you can achieve the goal individually by (carefully) changing N and a , and by comparing among $\frac{1}{2\pi}|G(s;a)|$, $|\hat{f}_j(a)|$, and $|\tilde{g}_j(a,N)|$.

分別隔離並評估三種誤差來源：Fourier coefficients truncation、sampling/aliasing，以及 non-periodicity/leakage。依 Q2(a) 的結果，

$\frac{1}{2\pi}|G(s;a)| = \frac{1}{2\sqrt{\pi a}}e^{-\pi^2 s^2/a}$, $|\hat{f}_j(a)| = \frac{1}{2\sqrt{\pi a}}e^{-j^2/(4a)}$, 且 $\hat{f}_j(a) = \frac{1}{2\pi}G\left(\frac{j}{2\pi};a\right)$. 其中 $\frac{1}{2\pi}|G(s;a)|$ 可視為連續頻譜包絡， $|\hat{f}_j(a)|$ 為 exact Fourier coefficients，而

$|\tilde{g}_j(a,N)|$ 則為在有限取樣點上以 FFT 計算出的 DFT coefficients。DFT 的兩個主要問題正是 aliasing 與 spectral leakage；其中 aliasing 來自有限取樣，而 leakage 則來自非週期資料在區間端點的不連續延拓

策略是透過改變 a 與 N ，並比較 $\frac{1}{2\pi}|G(s;a)|$, $|\hat{f}_j(a)|$, $|\tilde{g}_j(a,N)|$ 來分別突

顯三種誤差。若保持 a 足夠大，使 $g(\pm\pi;a) = e^{-a\pi^2}$ 極小，則 periodic extension 在端點近似連續，leakage 可被抑制；此時若再減少 N ，則高波數成分會摺疊回可解析波數範圍內，形成 aliasing。反之，若固定一個足夠大的 N 以減少 aliasing，再取較小的 a ，則 $g(x;a)$ 在 $x = \pm\pi$ 處不夠小，端點不連續便會造成 broad-band spectrum，也就是 leakage。至於 truncation，則可由 exact coefficients 在

$|j| > N/2$ 區域之外仍殘留多少能量來衡量。DFT 與 Fourier series 的關係可寫成

$\tilde{f}_j = \sum_{p=-\infty}^{\infty} \hat{f}_{j+Np}$, 因此 band 內的 DFT 係數除了真實 mode $\hat{f}_j$ 外，還包含被摺疊回

來的 aliased wavenumbers。

為此，選取三組參數：

$(a,N) = (20,32)$ for truncation, $(a,N) = (20,16)$ for aliasing, $(a,N) = (0.1,256)$ for leakage.

第一組以較大 a 壓低邊界值，同時保留有限但不大的高波數 tail，用來觀察 truncation；第二組固定同樣的 a 而減少 N ，以強化 sampling/aliasing；第三組則

固定很大的 N 而取很小的 a ，使 aliasing 幾乎可忽略，從而突顯 non-periodicity/leakage。

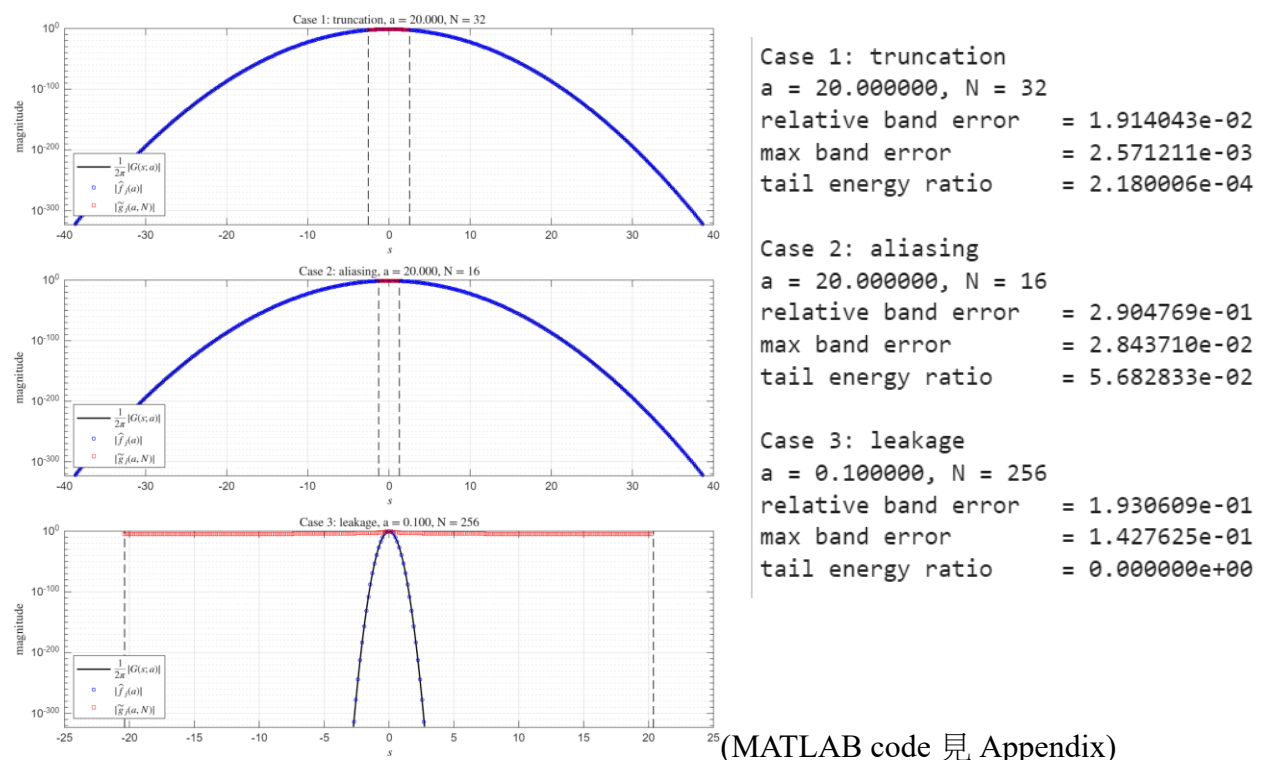
程式先用 Q2(a) 的解析式計算 $\frac{1}{2\pi}|G(s;a)|$ 與 $|\hat{f}_j(a)|$ ，再用 Q2(b) 的 FFT 程式計算

$|\tilde{g}_j(a, N)|$ 。此外，另外輸出三個量化指標：relative band error、max band error 與

tail energy ratio。前兩者用來衡量 $|\tilde{g}_j(a, N)|$ 與 $|\hat{f}_j(a)|$ 在可解析波數範圍內的差

異；tail energy ratio 則用來估計 exact coefficients 在 $|j| > N/2$ 之外尚存多少能量，

因此可作為 truncation 強弱的量化依據。



由圖與數值輸出可得，Case 1 ($a=20, N=32$)的

relative band error = 1.914043×10^{-2} , max band error = 2.571211×10^{-3} , 且 tail energy ratio = 2.180006×10^{-4} . 此時 a 足夠大，使 $g(\pm\pi; a)$ 幾乎為零，因此

leakage 很弱；同時 band 內 $|\tilde{g}_j(a, N)|$ 與 $|\hat{f}_j(a)|$ 幾乎重合，表示 aliasing 也不明

顯。故此 case 中最主要的誤差來源是：exact Fourier coefficients 在 cutoff 之外仍有一小段非零 tail，因此只能視為 Fourier coefficients truncation 所造成的誤差，而且其量級相對小。

Case 2($a=20, N=16$)中，

relative band error = 2.904769×10^{-1} , max band error = 2.843710×10^{-2} , 且

tail energy ratio = 5.682833×10^{-2} . 和 Case 1 相比， a 不變，但 N 減半，故 leakage

機制應幾乎相同；然而此時 tail energy ratio 顯著增加，代表 exact coefficients 在 Nyquist cutoff 之外仍有較多能量未被解析。依 $\tilde{f}_j = \sum_{p=-\infty}^{\infty} \hat{f}_{j+Np}$ ，這些高波數成分會摺疊回 band 內，使 $|\tilde{g}_j(a, N)|$ 與 $|\hat{f}_j(a)|$ 的差異明顯放大。因此此 case 主要隔離出 sampling/aliasing error。

Case 3 (a=0.1, N=256) 中，

relative band error = 1.930609×10^{-1} , max band error = 1.427625×10^{-1} , 而 tail energy ratio = 0. 由於 N=256 已相當大，若誤差仍顯著，則其主因不可能是 sampling/aliasing；另一方面，tail energy ratio 為零，表示 exact coefficients 幾乎完全落在 band 內，因此也不是 truncation 主導。此時真正的原因是 a 很小，使 $g(\pm\pi; 0.1) = e^{-0.1\pi^2} \approx 0.372$ 並不接近零，因此 $g(x; a)$ 在 $[-\pi, \pi]$ 上的 periodic

extension 於端點處不夠平滑，造成非常明顯的 leakage。圖中可見 $|\tilde{g}_j(a, N)|$ 在相當寬的波數範圍內維持顯著非零，與高度集中的 $|\hat{f}_j(a)|$ 形成強烈對比，這正是 broad-band spectrum 的典型特徵。

綜合而言，Q2(c) 顯示三種誤差可藉由控制 a 與 N 而被有效分離：當 a 大且 N 尚可時，誤差主要來自 truncation；在 a 固定且 N 降低時，sampling/aliasing 迅速增強；而當 N 很大但 a 很小時，non-periodicity/leakage 成為主導。

(d) We then want to isolate the additional aliasing error due to the nonlinear product. Let the nonlinear product be $h(x; a) = g^2(x; a) = e^{-2ax^2}$ and choose a set of N and a such that $\tilde{g}_j(a, N)$ is not aliased. One one hand, use the program developed in part (b) to evaluate the DFT coefficient of $h(x; a)$ on $x \in [-\pi, \pi]$ with $N + 1$ points. Called these DFT coefficient $\tilde{h}_{j,1}(a, N)$. On the other hand, implement the zero-padding technique with the 3/2 de-aliasing rule on $\tilde{g}_j(a, N)$ to calculate $\tilde{h}_{j,2}(a, N) = \sum_{m+m'=j} \tilde{g}_m(a, N) \tilde{g}_{m'}(a, N)$. By comparing between $|\tilde{h}_{j,1}(a, N)|$ and $|\tilde{h}_{j,2}(a, N)|$, isolate the additional aliasing error due to the nonlinear product.

以兩種方式計算其 DFT coefficients。第一種方式為直接將 $h(x; a)$ 在區間

$x \in [-\pi, \pi]$ 上以 N+1 個點取樣，再以 Q2(b) 的 FFT 程式計算 $\tilde{h}_{j,1}(a, N)$ 。第二種方

式則先取 $\tilde{g}_j(a, N)$ ，再使用 zero-padding 並配合 $M = \frac{3N}{2}$ 之 3/2 de-aliasing rule，於較大的頻譜範圍內完成 nonlinear product，最後切回原本的 N-mode band，得到 $\tilde{h}_{j,2}(a, N)$ 。

其理論背景如下。對 Fourier series 而言，若 $F(x) = u^2(x)$ ，則其 Fourier

coefficients 滿足卷積關係 $\hat{F}_j = \sum_{m+m'=j} \hat{u}_m \hat{u}_{m'}$ 。然而若改以 collocation/DFT 計算，則

會出現額外的 aliased terms， $\tilde{F}_j = \sum_{m+m'=j} \tilde{u}_m \tilde{u}_{m'} + \sum_{m+m'=j \pm N} \tilde{u}_m \tilde{u}_{m'}$ 。因此，即使 u 本身沒有 aliasing，其平方 u^2 仍可能產生額外 aliasing。對二次 nonlinear product 而言，可利用 $M = \frac{3N}{2}$ 的 zero-padding 技術將此額外 aliasing 移除。實作流程為：先將長度 N 的頻譜對稱補零至長度 M ，再做 inverse FFT 取得 expanded collocation points 上的 physical-space values，完成點對點平方後再做長度 M 的 FFT，最後只保留原本 $j \in I\left[-\frac{N}{2}, \frac{N}{2} - 1\right]$ 所對應的 modes。如此得到的 $\tilde{h}_{j,2}(a, N)$ 可視為 de-aliased 結果。若比較 $|\tilde{h}_{j,1}(a, N)|$ 與 $|\tilde{h}_{j,2}(a, N)|$ ，則兩者之差異即可視為由 nonlinear product 所造成的 additional aliasing error。為了使此比較能專注於 nonlinear product 本身，而非混入 $g(x; a)$ 本身的 sampling/aliasing 或 leakage，先選取一組使 $\tilde{g}_j(a, N)$ 幾乎不 aliased 的參數：

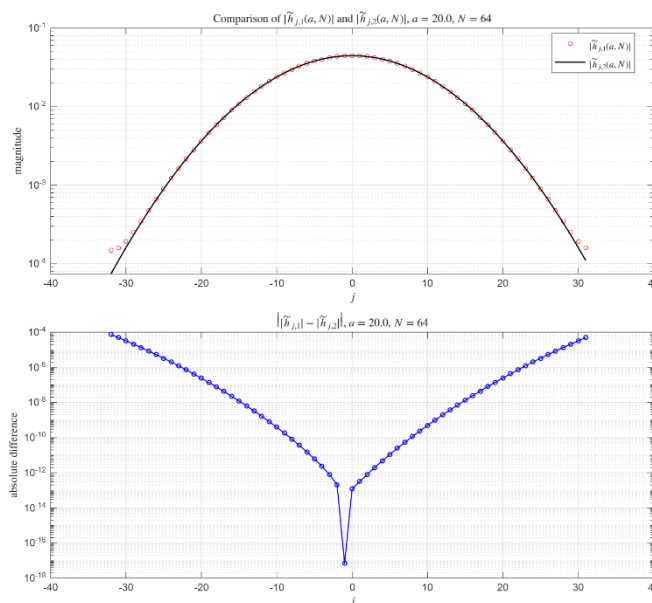
$a = 20, N = 64, M = \frac{3N}{2} = 96$ 。此時 $g(x; a) = e^{-ax^2}$ 在 $x = \pm\pi$ 已極小，因此 leakage

很弱；同時 $N=64$ 也足以解析 g 的主要頻譜。程式輸出顯示

g relative band error = 1.004280×10^{-6} , g max band error = 1.741449×10^{-7} ，表示

$\tilde{g}_j(a, N)$ 與 exact band-limited spectrum 已非常接近，因此後續 $\tilde{h}_{j,1}$ 與 $\tilde{h}_{j,2}$ 的差異，

可合理歸因於 nonlinear product 所產生的額外 aliasing，而非來自 g 本身。



Command Window					
j	real_h_tilde_1	real_h_tilde_2	abs_h_tilde_1	abs_h_tilde_2	abs_difference
-12	0.018134	0.018134	0.018134	0.018134	1.6487e-09
-11	0.020938	0.020938	0.020938	0.020938	8.1111e-10
-10	0.023874	0.023874	0.023874	0.023874	3.8966e-10
-9	0.026885	0.026885	0.026885	0.026885	1.8235e-10
-8	0.029898	0.029898	0.029898	0.029898	8.2873e-11
-7	0.032837	0.032837	0.032837	0.032837	3.6418e-11
-6	0.035616	0.035616	0.035616	0.035616	1.5377e-11
-5	0.038151	0.038151	0.038151	0.038151	6.1772e-12
-4	0.040359	0.040359	0.040359	0.040359	2.3204e-12
-3	0.042163	0.042163	0.042163	0.042163	7.8566e-13
-2	0.043502	0.043502	0.043502	0.043502	2.1162e-13
-1	0.044325	0.044325	0.044325	0.044325	6.9389e-18
0	0.044603	0.044603	0.044603	0.044603	1.2131e-13
1	0.044325	0.044325	0.044325	0.044325	3.2045e-13
2	0.043502	0.043502	0.043502	0.043502	8.0645e-13
3	0.042163	0.042163	0.042163	0.042163	1.9904e-12
4	0.040359	0.040359	0.040359	0.040359	4.761e-12
5	0.038151	0.038151	0.038151	0.038151	1.1024e-11
6	0.035616	0.035616	0.035616	0.035616	2.4776e-11
7	0.032837	0.032837	0.032837	0.032837	5.4198e-11
8	0.029898	0.029898	0.029898	0.029898	1.1568e-10
9	0.026885	0.026885	0.026885	0.026885	2.4139e-10
10	0.023874	0.023874	0.023874	0.023874	4.9326e-10
11	0.020938	0.020938	0.020938	0.020938	9.8845e-10
12	0.018134	0.018134	0.018134	0.018134	1.9447e-09

a = 20.000000
N = 64
M = 96
g relative band error = 1.004280e-06
g max band error = 1.741449e-07
relative nonlinear alias error = 6.654997e-04
max nonlinear alias error = 7.408864e-05

數值結果為

relative nonlinear alias error = 6.654997×10^{-4} , max nonlinear alias error = 7.408864×10^{-5} 。

由上圖可見，在大部分低至中等波數範圍內， $|\tilde{h}_{j,1}(a, N)|$ 與 $|\tilde{h}_{j,2}(a, N)|$ 幾乎完全

重合，表示 nonlinear product 所造成的額外 aliasing 對主頻譜核心部分影響很小。然而在接近 band edge 的高波數區域，兩者開始出現可辨識差異，且下方誤差圖顯示絕對差異隨 $|j|$ 增加而明顯上升，在 $j \approx \pm 31$ 附近最大。這表示 nonlinear product 所新生成的較高波數成分，最先污染的正是靠近 cutoff 的 modes；而這些成分在未做 de-aliasing 時，會摺疊回原本 band 內，形成額外 aliasing。此結果與理論相符。雖然 g 本身在 $N=64$ 下幾乎不 aliased，但 $h = g^2$ 的頻譜範圍比 g 更寬，因此直接在原本的 N -point collocation grid 上計算 product 時，會產生額外的高波數資訊，並折返至 band 內；相對地，使用 $M = \frac{3N}{2}$ 的 zero-padding 後，這些新增的高波數成分可先被容納於擴張後的頻譜空間中，再於最後切回原 band，因此能有效移除額外 aliasing。由於所選參數已使 g 本身的 aliasing 幾乎可以忽略，故圖中 $|\tilde{h}_{j,1}(a, N)| - |\tilde{h}_{j,2}(a, N)|$ 所呈現的差異，便可視為 nonlinear product 導致的 additional aliasing error 之直接證據。

Q3(a): 考慮 $f(x) = 2 \sin x - \frac{1}{4} e^x - 1$. 其解析導數為 $f'(x) = 2 \cos x - \frac{1}{4} e^x$. 因此，本題可直接以 exact derivative 作為基準，評估不同 numerical differentiation schemes 的誤差。

選擇兩種 finite difference。第一種為 forward difference： $f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{h}$ ，其截斷誤差為 $O(h)$ 。第二種為 three-point central difference：

$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}$ ，其截斷誤差為 $O(h^2)$ 。因此，理論上 central difference 應

比 forward difference 收斂更快，且在相同 h 下具有較小誤差。對於端點，分別使用一階 backward difference 與二階 one-sided difference。

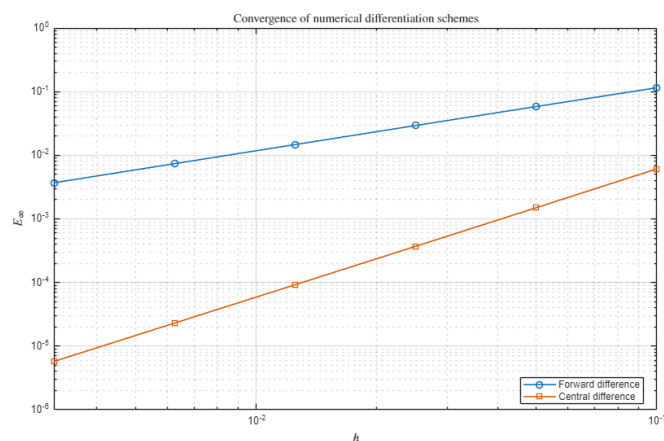
在區間 $[-1, 1]$ 上進行，令 $x_i = -1 + ih$, $i = 0, 1, \dots, N-1$, $h = \frac{2}{N-1}$ ，並使用

$N = 21, 41, 81, 161, 321, 641$ 六組網格。誤差以 maximum norm 定義為

$$E_\infty = \max_i |f'_i{}^{\text{num}} - f'(x_i)|.$$

Maximum error of the two numerical differentiation schemes

N	h	E_∞ (Forward)	E_∞ (Central)
21	0.1	1.1515×10^{-1}	6.1125×10^{-3}
41	0.05	5.8324×10^{-2}	1.4978×10^{-3}
81	0.025	2.9347×10^{-2}	3.7060×10^{-4}
161	0.0125	1.4720×10^{-2}	9.2165×10^{-5}
321	0.00625	7.3714×10^{-3}	2.2980×10^{-5}
641	0.003125	3.6886×10^{-3}	5.7374×10^{-6}



由表可見，當 h 每減半一次，forward difference 的 maximum error 約減半，而 central difference 的誤差約降為原來的四分之一。由相鄰兩組網格估計之 observed order 分別為：

$p_{\text{forward}} = 0.981, 0.991, 0.995, 0.998, 0.999$, $p_{\text{central}} = 2.029, 2.015, 2.008, 2.004, 2.002$.
這與理論上完全一致。特別是在最細網格 $N=641$ 時，

$E_{\infty}^{\text{forward}} = 3.688560 \times 10^{-3}$, $E_{\infty}^{\text{central}} = 5.737387 \times 10^{-6}$, 顯示 central difference 的誤差已比 forward difference 小約三個數量級。

圖中兩條誤差曲線近似為直線，表示誤差隨 h 呈冪次下降。Forward difference 的斜率約為 1，而 central difference 的斜率約為 2，再次驗證前者為一階、後者為二階。此外，central difference 的曲線始終位於 forward difference 之下，表示在相同網格解析度下，其精度顯著較高。

Q3(b): 計算 $I = \int_{-1}^1 f(x) dx$, $f(x) = 2 \sin x - \frac{1}{4} e^x - 1$, 並與 exact solution 比較。選擇

2-point Gauss-Legendre quadrature 與 composite Simpson's 1/3 rule。

首先，exact integral 可由解析積分直接求得：

$$I_{\text{exact}} = \int_{-1}^1 \left(2 \sin x - \frac{1}{4} e^x - 1 \right) dx = \left[-2 \cos x - \frac{1}{4} e^x - x \right]_{-1}^1 = -2 - \frac{e - e^{-1}}{4} \approx -2.587600596821900.$$

對於 Gaussian quadrature，在 $[-1, 1]$ 上的 2-point Gauss-Legendre formula 為

$$I_{G2} = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right), \text{此方法對所有三次以下多項式皆為精確。}$$

對於 composite Simpson's 1/3 rule，若在 $[-1, 1]$ 上取等距點 $x_i = -1 + ih$, $h = \frac{2}{N-1}$,

$$\text{且 } N-1 \text{ 為偶數，則 } I_s = \frac{h}{3} \left[f(x_0) + f(x_{N-1}) + 4 \sum_{k=1}^{(N-1)/2} f(x_{2k-1}) + 2 \sum_{k=1}^{(N-3)/2} f(x_{2k}) \right]. \text{其總誤}$$

差為 $O(h^4)$ ，因此理論上當 h 減半時，Simpson's rule 的誤差應約縮小為原來的 1/16。

以 MATLAB 計算所得結果如下。對 2-point Gauss-Legendre，

$$I_{G2} = -2.585674021977432, \quad |I_{G2} - I_{\text{exact}}| = 1.926574844468121 \times 10^{-3}.$$

對 composite Simpson's 1/3 rule，取多組不同網格點數後得到：

N	h	I_s	$ I_s - I_{\text{exact}} $
3	1	-2.5905	2.9128×10^{-3}
5	0.5	-2.5878	1.9811×10^{-4}
9	0.25	-2.5876	1.2657×10^{-5}
17	0.125	-2.5876	7.9550×10^{-7}
33	0.0625	-2.5876	4.9788×10^{-8}
65	0.03125	-2.5876	3.1129×10^{-9}
129	0.015625	-2.5876	1.9457×10^{-10}

由相鄰兩組網格誤差估計之 observed order 為 3.878,3.9682,3.992,3.998,3.9995,3.9999,可見其收斂階數非常穩定地趨近 4,與理論上的 $O(h^4)$ 完全一致。最細網格 $N=129$ 時，

$I_s = -2.587600597016471$, $|I_s - I_{\text{exact}}| = 1.945705818684473 \times 10^{-10}$, 已達到極高精度。

綜合比較可知，2-point Gauss-Legendre quadrature 的優點在於：僅用兩個內點便得到不錯的近似值，其誤差約為 10^{-3} 。這反映了 Gaussian quadrature 較高的 polynomial exactness。另一方面，composite Simpson's 1/3 rule 雖屬 difference formula method，但透過網格加密可穩定呈 $O(h^4)$ 收斂；在 $N=129$ 時，其誤差已降至 10^{-10} 等級，遠小於 2-point Gaussian quadrature。故若只允許極少量函數評估，2-point Gauss-Legendre 較有效率；若允許細化網格，則 composite Simpson's 1/3 rule 可達到更高的精度。

Q3(c): 以至少兩種不同收斂階數的 root-searching schemes，找出在 $x=-5$ 附近最近的兩個零點，並比較其 accuracy。選擇 bisection method 與 Newton-Raphson method。依講義，bisection method 的收斂階數為 $\alpha = 1$, 而 Newton-Raphson method 在 simple root 且初始值足夠接近時具有 $\alpha = 2$ 之局部二次收斂。

Bisection method 的迭代方式為：若已知區間 $[a,b]$ 滿足 $f(a)f(b) < 0$, 則取中點

$c = \frac{a+b}{2}$, 並依 $f(a)f(c)$ 與 $f(c)f(b)$ 的符號決定新的子區間。其誤差會隨每次二分

而約縮小一半，因此為 first-order convergence。Newton-Raphson method 則使用

$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$, 其中 $f'(x) = 2 \cos x - \frac{1}{4}e^x$ 。若迭代已進入根附近，則誤差滿足

$|e_{n+1}| \approx C|e_n|^2$, 因此理論上應顯著快於 bisection。

實作上，先在 $[-10,0]$ 內進行掃描，找出所有 sign-change intervals，再依區間中點與 -5 的距離排序，選取最近的兩個 brackets。MATLAB 所找到的最近兩個 sign-change intervals 為 $[-5.759150000000000, -5.759125000000000]$, 以及 $[-3.668900000000000, -3.668875000000000]$ 。

之後，對每個 interval 分別使用 bisection method 與 Newton-Raphson method 求解，並以高精度 reference root 作為誤差基準。數值結果如下。

Comparison of bisection method and Newton-Raphson method for the two closest roots near $x = -5$.

Root	Reference root	Bisection root	Newton root	Iter. (Bis.)	Iter. (Newt.)
1	-5.75913125264825	-5.75913125264825	-5.75913125264825	30	2
2	-3.66887670271014	-3.66887670271014	-3.66887670271014	27	2

兩種方法皆成功找到與 reference root 幾乎一致的近似值。對第一個零點，其

absolute errors 分別為 $|x_{\text{bis}} - x_{\text{ref}}| = 8.8818 \times 10^{-16}$, $|x_{\text{newt}} - x_{\text{ref}}| = 8.8818 \times 10^{-16}$, 對第

二個零點則為 $|x_{\text{bis}} - x_{\text{ref}}| = 2.6645 \times 10^{-15}$, $|x_{\text{newt}} - x_{\text{ref}}| = 0$. 可見兩者最終都已很精

確，因此在最終 root accuracy 上差異極小；真正的差異主要反映在收斂速度上。

從迭代次數可清楚看出，bisection method 分別花費 30 與 27 次迭代，而

Newton-Raphson method 僅需 2 次迭代便達到同等精度。這與理論完全一致：

bisection method 為 first-order convergence，誤差只會線性縮小；Newton-

Raphson method 則為 second-order convergence，一旦初始值已落在 root 附近，

誤差會以平方速度下降，因此收斂極快。由於本題的 Newton 初始值取自已經

很窄的 sign-change interval 中點，起始點本身就非常接近根，故只需兩步便已

很精確

程式另外輸出了 observed order estimates，但結果分別為 bisection 的 -17.12 與

3.1586，以及 Newton 的 NaN。此現象是因為根已在極少步內達到 machine

precision，後期誤差不再遵循理想的 asymptotic scaling。對 Newton-Raphson

method 而言，只做兩步便已收斂，故根本沒有足夠的晚期誤差點可用來穩定估

計 observed order；對 bisection 而言，最後幾步誤差也已接近 floating-point

round-off level，因此以最後三步誤差估算收斂階時會受到嚴重污染。故本題判

斷收斂階數時，應以理論與整體迭代行為為主，而不應將這些末期 numerical

estimates 視為可靠的階數量測。

Appendix

本作業之 MATLAB 程式碼撰寫過程中，我有使用 AI 作為輔助工具，主要用途包括：協助整理數值方法的程式架構、提供 MATLAB 語法建議、協助圖形排版與除錯，以及將不同題目需求整理成較完整的程式流程。

Q1(a)

prompts：請給我 MATLAB 程式碼，用等距節點做 polynomial interpolation，並分析 error distribution。

把 MATLAB 程式碼整理成可直接執行的版本，並修正 legend、繪圖與輸出時的錯誤。

```
clear; clc; close all;

f = @(x) cos(10*x.^2) .* exp(-x);

n_list = [5 9 13 17 21 25 33 41 49];

Nfine = 5001;

xfine = linspace(-1, 1, Nfine).';

ytrue = f(xfine);

maxErr = zeros(size(n_list));

rmsErr = zeros(size(n_list));

xAtMax = zeros(size(n_list));

set(groot, ...

    'defaultAxesFontName', 'Times New Roman', ...

    'defaultTextFontName', 'Times New Roman', ...

    'defaultAxesFontSize', 12, ...

    'defaultAxesLinewidth', 0.9, ...

    'defaultLineLinewidth', 1.6, ...

    'defaultFigureColor', 'w');

cTrue = [0.10, 0.10, 0.10];
```

```

cInterp = [0.85, 0.20, 0.20];

cNode    = [0.10, 0.35, 0.85];

cErr      = [0.10, 0.25, 0.75];

fig1 = figure('Units','pixels','Position',[80 60 1450 900],'Color','w');

t11 = tiledlayout(3,3,'TileSpacing','compact','Padding','compact');

for k = 1:length(n_list)

    n = n_list(k);

    xnodes = linspace(-1,1,n).';

    ynodes = f(xnodes);

    a = newton_divided_diff(xnodes, ynodes);

    yinterp = newton_eval(a, xnodes, xfine);

    err = yinterp - ytrue;

    [maxErr(k), idx] = max(abs(err));

    rmsErr(k) = sqrt(mean(err.^2));

    xAtMax(k) = xfine(idx);

    ax = nexttile;

    hold(ax,'on');

    h1 = plot(ax, xfine, ytrue, '-', 'color', cTrue, 'Linewidth', 1.8);

    h2 = plot(ax, xfine, yinterp, '--', 'color', cInterp, 'Linewidth', 1.6);

    h3 = plot(ax, xnodes, ynodes, 'o', 'color', cNode, 'MarkerFaceColor', cNode,
'MarkerSize', 3.5, 'LineStyle', 'none');

    if k == 1

        legend(ax, [h1 h2 h3], {'True function','Interpolation','Nodes'},
'Location','best', 'Box','off', 'FontSize',8);

    end

```

```

grid(ax, 'on');

ax.GridAlpha = 0.18;

ax.MinorGridAlpha = 0.08;

ax.XMinorGrid = 'on';

ax.YMinorGrid = 'on';

ax.Box = 'on';

xlim(ax, [-1 1]);

xlabel(ax, 'x');

ylabel(ax, 'y');

title(ax, sprintf('n = %d, degree = %d', n, n-1),
'FontWeight', 'bold', 'FontSize', 11);

end

title(tl1, 'Q1(a): Function and Polynomial Interpolation',
'FontSize', 16, 'FontWeight', 'bold');

fig2 = figure('Units', 'pixels', 'Position', [100 80 1450 900], 'Color', 'w');

tl2 = tiledlayout(3,3, 'TileSpacing', 'compact', 'Padding', 'compact');

for k = 1:length(n_list)

    n = n_list(k);

    xnodes = linspace(-1,1,n).';

    ynodes = f(xnodes);

    a = newton_divided_diff(xnodes, ynodes);

    yinterp = newton_eval(a, xnodes, xfine);

    err = yinterp - ytrue;

    ax = nexttile;

    hold(ax, 'on');

```

```

    plot(ax, xfine, err, '-', 'color', cErr, 'Linewidth', 1.5);

    yline(ax, 0, '--', 'color', [0.35 0.35 0.35], 'Linewidth', 1.0);

    grid(ax, 'on');

    ax.GridAlpha = 0.18;

    ax.MinorGridAlpha = 0.08;

    ax.XMinorGrid = 'on';

    ax.YMinorGrid = 'on';

    ax.Box = 'on';

    xlim(ax, [-1 1]);

    xlabel(ax, 'x');

    ylabel(ax, 'e_n(x)');

    title(ax, sprintf('Error, n = %d', n), 'FontWeight', 'bold', 'FontSize', 11);

end

title(tl2, 'Q1(a): Error Distribution', 'FontSize', 16, 'FontWeight', 'bold');

fig3 = figure('Units', 'pixels', 'Position', [220 120 900 620], 'Color', 'w');

ax3 = axes(fig3);

hold(ax3, 'on');

semilogy(ax3, n_list, maxErr, '-o', 'color', [0.10 0.40 0.80], 'MarkerFaceColor',
[0.10 0.40 0.80], 'MarkerSize', 6.5, 'Linewidth', 1.8);

grid(ax3, 'on');

ax3.GridAlpha = 0.20;

ax3.MinorGridAlpha = 0.08;

ax3.XMinorGrid = 'on';

ax3.YMinorGrid = 'on';

ax3.Box = 'on';

```



```

xlabel(ax3, 'Number of interpolation points n', 'FontSize', 13);

ylabel(ax3, 'max |p_{n-1}(x) - f(x)|', 'FontSize', 13);

title(ax3, 'Maximum Absolute Error on the Fine Grid',
'FontSize',16,'FontWeight','bold');

n_zoom = n_list(end);

xnodes = linspace(-1,1,n_zoom).';

ynodes = f(xnodes);

a = newton_divided_diff(xnodes, ynodes);

yinterp = newton_eval(a, xnodes, xfine);

err = yinterp - ytrue;

fig4 = figure('Units','pixels','Position',[180 130 1200 560],'Color','w');

t14 = tiledlayout(1,2,'TileSpacing','compact','Padding','compact');

ax41 = nexttile;

plot(ax41, xfine, err, '-', 'Color', cErr, 'Linewidth', 1.5); hold(ax41,'on');

yline(ax41, 0, '--', 'Color', [0.35 0.35 0.35], 'Linewidth', 1.0);

xlim(ax41, [-1, -0.7]);

grid(ax41,'on');

ax41.GridAlpha = 0.18;

ax41.XMinorGrid = 'on';

ax41.YMinorGrid = 'on';

ax41.Box = 'on';

xlabel(ax41,'x');

ylabel(ax41,'e_n(x)');

title(ax41, sprintf('Left endpoint zoom, n = %d', n_zoom),
'FontWeight','bold','FontSize',12);

```

```

ax42 = nexttile;

plot(ax42, xfine, err, '-', 'Color', cErr, 'Linewidth', 1.2); hold(ax42,'on');

yline(ax42, 0, '--', 'Color', [0.35 0.35 0.35], 'Linewidth', 1.0);

xlim(ax42, [0.7, 1]);

grid(ax42,'on');

ax42.GridAlpha = 0.18;

ax42.XMinorGrid = 'on';

ax42.YMinorGrid = 'on';

ax42.Box = 'on';

xlabel(ax42,'x');

ylabel(ax42,'e_n(x)');

title(ax42, sprintf('Right endpoint zoom, n = %d', n_zoom),
'FontWeight','bold','FontSize',12);

title(tl4, 'Q1(a): Endpoint Error Zoom', 'FontSize',16,'FontWeight','bold');

ResultTable = table(n_list(:), (n_list(:)-1), maxErr(:), rmsErr(:), xAtMax(:),
'VariableNames', {'n_points','degree','max_abs_error','rms_error','x_at_max_error'});

disp('=====');

disp('Summary of Q1(a): polynomial interpolation with evenly spaced points');

disp(ResultTable);

disp('=====');

function a = newton_divided_diff(x, y)

    n = length(x);

    a = y(:);

    for j = 2:n

        a(j:n) = (a(j:n) - a(j-1:n-1)) ./ (x(j:n) - x(1:n-j+1));

```

```

        end

    end

function p = newton_eval(a, xnodes, xq)

    n = length(a);

    p = a(n) * ones(size(xq));

    for k = n-1:-1:1

        p = a(k) + (xq - xnodes(k)) .* p;

    end

end

```

Q1(b)

prompts：請用 Chebyshev points 重寫 Q1(b) 的 MATLAB 程式碼，並保持與 Q1(a) 相同的整體架構。

```

clear;clc;close all;

f=@(x) cos(10*x.^2).*exp(-x);

n_list=[5 9 13 17 21 25 33 41 49];

Nfine=5001;

xfine=linspace(-1,1,Nfine).';

ytrue=f(xfine);

maxErr=zeros(size(n_list));

rmsErr=zeros(size(n_list));

xAtMax=zeros(size(n_list));

set(groot,'defaultAxesFontName','Times New Roman','defaultTextFontName','Times New
Roman','defaultAxesFontSize',12,'defaultAxesLinewidth',0.9,'defaultLineLinewidth',1.6,
'defaultFigureColor','w');

cTrue=[0.10,0.10,0.10];

cInterp=[0.85,0.20,0.20];

```

```

cNode=[0.10,0.35,0.85];

cErr=[0.10,0.25,0.75];

fig1=figure('Units','pixels','Position',[80 60 1450 900],'Color','w');

t11=tildeLayout(3,3,'TileSpacing','compact','Padding','compact');

for k=1:length(n_list)

    n=n_list(k);

    j=(1:n).';

    xnodes=-cos((2*j-1)*pi/(2*n));

    ynodes=f(xnodes);

    a=newton_divided_diff(xnodes,ynodes);

    yinterp=newton_eval(a,xnodes,xfine);

    err=yinterp-ytrue;

    [maxErr(k),idx]=max(abs(err));

    rmsErr(k)=sqrt(mean(err.^2));

    xAtMax(k)=xfine(idx);

    ax=nexttile;

    hold(ax,'on');

    h1=plot(ax,xfine,ytrue,'-','Color',cTrue,'Linewidth',1.8);

    h2=plot(ax,xfine,yinterp,'--','Color',cInterp,'Linewidth',1.6);

    h3=plot(ax,xnodes,ynodes,'o','Color',cNode,'MarkerFaceColor',cNode,'MarkerSize',3.5,'LineStyle','none');

    if k==1

        legend(ax,[h1 h2 h3],{'True
function','Interpolation','Nodes'},'Location','best','Box','off','FontSize',8);

```

```

end

grid(ax,'on');

ax.GridAlpha=0.18;

ax.MinorGridAlpha=0.08;

ax.XMinorGrid='on';

ax.YMinorGrid='on';

ax.Box='on';

xlim(ax,[-1 1]);

xlabel(ax,'x');

ylabel(ax,'y');

title(ax,sprintf('n = %d, degree = %d',n,n-1),'FontWeight','bold','FontSize',11);

end

title(tl1,'Q1(b): Function and Polynomial Interpolation (Chebyshev
Nodes)','FontSize',16,'FontWeight','bold');

fig2=figure('Units','pixels','Position',[100 80 1450 900],'Color','w');

tl2=tiledlayout(3,3,'TileSpacing','compact','Padding','compact');

for k=1:length(n_list)

    n=n_list(k);

    j=(1:n).';

    xnodes=-cos((2*j-1)*pi/(2*n));

    ynodes=f(xnodes);

    a=newton_divided_diff(xnodes,ynodes);

    yinterp=newton_eval(a,xnodes,xfine);

    err=yinterp-ytrue;

    ax=nexttile;

```

```

hold(ax, 'on');

plot(ax,xfine,err,'-','color',cErr,'Linewidth',1.5);

yline(ax,0,'--','color',[0.35 0.35 0.35],'Linewidth',1.0);

grid(ax,'on');

ax.GridAlpha=0.18;

ax.MinorGridAlpha=0.08;

ax.XMinorGrid='on';

ax.YMinorGrid='on';

ax.Box='on';

xlim(ax,[-1 1]);

xlabel(ax,'x');

ylabel(ax,'e_n(x)');

title(ax,sprintf('Error, n = %d',n),'FontWeight','bold','FontSize',11);

end

title(tl2,'Q1(b): Error Distribution (Chebyshev
Nodes)','FontSize',16,'FontWeight','bold');

fig3=figure('Units','pixels','Position',[220 120 900 620],'color','w');

ax3=axes(fig3);

hold(ax3,'on');

semilogy(ax3,n_list,maxErr,'-o','color',[0.10 0.40 0.80],'MarkerFaceColor',[0.10 0.40
0.80],'MarkerSize',6.5,'Linewidth',1.8);

grid(ax3,'on');

ax3.GridAlpha=0.20;

ax3.MinorGridAlpha=0.08;

ax3.XMinorGrid='on';

```



```

ax3.YMinorGrid='on';

ax3.Box='on';

xlabel(ax3,'Number of interpolation points n','FontSize',13);

ylabel(ax3,'max |p_{n-1}(x) - f(x)|','FontSize',13);

title(ax3,'Maximum Absolute Error on the Fine Grid (Chebyshev
Nodes)','FontSize',16,'Fontweight','bold');

n_zoom=n_list(end);

j=(1:n_zoom).';

xnodes=-cos((2*j-1)*pi/(2*n_zoom));

ynodes=f(xnodes);

a=newton_divided_diff(xnodes,ynodes);

yinterp=newton_eval(a,xnodes,xfine);

err=yinterp-ytrue;

fig4=figure('Units','pixels','Position',[180 130 1200 560],'Color','w');

t14=tilayout(1,2,'TileSpacing','compact','Padding','compact');

ax41=nexttile;

plot(ax41,xfine,err,'-','Color',cErr,'Linewidth',1.5);hold(ax41,'on');

yline(ax41,0,'--','Color',[0.35 0.35 0.35],'Linewidth',1.0);

xlim(ax41,[-1,-0.7]);

grid(ax41,'on');

ax41.GridAlpha=0.18;

ax41.XMinorGrid='on';

ax41.YMinorGrid='on';

ax41.Box='on';

xlabel(ax41,'x');

```

```

ylabel(ax41, 'e_n(x)');

title(ax41,sprintf('Left endpoint zoom, n =
%d',n_zoom), 'FontWeight','bold','FontSize',12);

ax42=nexttile;

plot(ax42,xfine,err,'-','Color',cErr,'Linewidth',1.2);hold(ax42,'on');

yline(ax42,0,'--','Color',[0.35 0.35 0.35],'Linewidth',1.0);

xlim(ax42,[0.7,1]);

grid(ax42,'on');

ax42.GridAlpha=0.18;

ax42.XMinorGrid='on';

ax42.YMinorGrid='on';

ax42.Box='on';

xlabel(ax42, 'x');

ylabel(ax42, 'e_n(x)');

title(ax42,sprintf('Right endpoint zoom, n =
%d',n_zoom), 'FontWeight','bold','FontSize',12);

title(tl4,'Q1(b): Endpoint Error Zoom (Chebyshev
Nodes)','FontSize',16,'FontWeight','bold');

ResultTable=table(n_list(:),(n_list(:)-
1),maxErr(:),rmsErr(:),xAtMax(:),'VariableNames',{'n_points','degree','max_abs_error',
'rms_error','x_at_max_error'});

disp('=====');

disp('Summary of Q1(b): polynomial interpolation with Chebyshev points');

disp(ResultTable);

disp('=====');

```

```

function a=newton_divided_diff(x,y)

n=length(x);

a=y(:);

for j=2:n

    a(j:n)=(a(j:n)-a(j-1:n-1))./(x(j:n)-x(1:n-j+1));

end

end

function p=newton_eval(a,xnodes,xq)

n=length(a);

p=a(n)*ones(size(xq));

for k=n-1:-1:1

    p=a(k)+(xq-xnodes(k)).*p;

end

end

```

Q1(c)

prompts：請實作 cubic spline interpolation，並盡量使用與講義一致的 notation。

```

clear;clc;close all;

f=@(x) cos(10*x.^2).*exp(-x);

n_list=[5 9 13 17 21 25 33 41 49];

Nfine=5001;

xfine=linspace(-1,1,Nfine).';

ytrue=f(xfine);

maxErr=zeros(size(n_list));

rmsErr=zeros(size(n_list));

xAtMax=zeros(size(n_list));

```

```

set(groot,'defaultAxesFontName','Times New Roman','defaultTextFontName','Times New
Roman','defaultAxesFontSize',12,'defaultAxesLinewidth',0.9,'defaultLineLinewidth',1.6,
'defaultFigureColor','w');

cTrue=[0.10,0.10,0.10];

cInterp=[0.85,0.20,0.20];

cNode=[0.10,0.35,0.85];

cErr=[0.10,0.25,0.75];

fig1=figure('Units','pixels','Position',[80 60 1450 900],'Color','w');

t1l=tiledlayout(3,3,'TileSpacing','compact','Padding','compact');

for k=1:length(n_list)

    n=n_list(k);

    x=linspace(-1,1,n).';

    y=f(x);

    fpp=natural_spline_fpp(x,y);

    S4f=spline_eval(x,y,fpp,xfine);

    err=S4f-ytrue;

    [maxErr(k),idx]=max(abs(err));

    rmsErr(k)=sqrt(mean(err.^2));

    xAtMax(k)=xfine(idx);

    ax=nexttile;

    hold(ax,'on');

    h1=plot(ax,xfine,ytrue,'-','Color',cTrue,'Linewidth',1.8);

    h2=plot(ax,xfine,S4f,'--','Color',cInterp,'Linewidth',1.6);

    h3=plot(ax,x,y,'o','Color',cNode,'MarkerFaceColor',cNode,'MarkerSize',3.5,'LineStyle',

```

```

'none');

    if k==1

        legend(ax,[h1 h2 h3],{'True function','Spline
interpolation','Nodes'},'Location','best','Box','off','FontSize',8);

    end

    grid(ax,'on');

    ax.GridAlpha=0.18;

    ax.MinorGridAlpha=0.08;

    ax.XMinorGrid='on';

    ax.YMinorGrid='on';

    ax.Box='on';

    xlim(ax,[-1 1]);

    xlabel(ax,'x');

    ylabel(ax,'y');

    title(ax,sprintf('n = %d',n),'FontWeight','bold','FontSize',11);

end

title(tl1,'Q1(c): Function and Cubic Spline
Interpolation','FontSize',16,'FontWeight','bold');

fig2=figure('Units','pixels','Position',[100 80 1450 900],'color','w');

tl2=tildeLayout(3,3,'TileSpacing','compact','Padding','compact');

for k=1:length(n_list)

    n=n_list(k);

    x=linspace(-1,1,n).';

    y=f(x);

    fpp=natural_spline_fpp(x,y);

```

```

s4f=spline_eval(x,y,fpp,xfine);

err=S4f-ytrue;

ax=nexttile;

hold(ax,'on');

plot(ax,xfine,err,'-','Color',cErr,'Linewidth',1.5);

yline(ax,0,'--','Color',[0.35 0.35 0.35],'Linewidth',1.0);

grid(ax,'on');

ax.GridAlpha=0.18;

ax.MinorGridAlpha=0.08;

ax.XMinorGrid='on';

ax.YMinorGrid='on';

ax.Box='on';

xlim(ax,[-1 1]);

xlabel(ax,'x');

ylabel(ax,'e_n(x)');

title(ax,sprintf('Error, n = %d',n),'FontWeight','bold','FontSize',11);

end

title(tl2,'Q1(c): Error Distribution (Cubic
Spline)','FontSize',16,'FontWeight','bold');

fig3=figure('Units','pixels','Position',[220 120 900 620],'color','w');

ax3=axes(fig3);

hold(ax3,'on');

semilogy(ax3,n_list,maxErr,'-o','Color',[0.10 0.40 0.80],'MarkerFaceColor',[0.10 0.40
0.80],'MarkerSize',6.5,'Linewidth',1.8);

grid(ax3,'on');

```

```

ax3.GridAlpha=0.20;

ax3.MinorGridAlpha=0.08;

ax3.XMinorGrid='on';

ax3.YMinorGrid='on';

ax3.Box='on';

xlabel(ax3,'Number of interpolation points n','FontSize',13);

ylabel(ax3,'max |S_4f(x)-f(x)|','FontSize',13);

title(ax3,'Maximum Absolute Error on the Fine Grid (Cubic
Spline)','FontSize',16,'FontWeight','bold');

n_zoom=n_list(end);

x=linspace(-1,1,n_zoom).';

y=f(x);

fpp=natural_spline_fpp(x,y);

s4f=spline_eval(x,y,fpp,xfine);

err=s4f-ytrue;

fig4=figure('Units','pixels','Position',[180 130 1200 560],'Color','w');

t14=tiledlayout(1,2,'TileSpacing','compact','Padding','compact');

ax41=nexttile;

plot(ax41,xfine,err,'-','Color',cErr,'Linewidth',1.5);hold(ax41,'on');

yline(ax41,0,'--','Color',[0.35 0.35 0.35],'Linewidth',1.0);

xlim(ax41,[-1,-0.7]);

grid(ax41,'on');

ax41.GridAlpha=0.18;

ax41.XMinorGrid='on';

ax41.YMinorGrid='on';

```

```

ax41.Box='on';

xlabel(ax41,'x');

ylabel(ax41,'e_n(x)');

title(ax41,sprintf('Left endpoint zoom, n =
%d',n_zoom),'FontWeight','bold','FontSize',12);

ax42=nexttile;

plot(ax42,xfine,err,'-','Color',cErr,'Linewidth',1.2);hold(ax42,'on');

ylines(ax42,0,'--','Color',[0.35 0.35 0.35],'Linewidth',1.0);

xlim(ax42,[0.7,1]);

grid(ax42,'on');

ax42.GridAlpha=0.18;

ax42.XMinorGrid='on';

ax42.YMinorGrid='on';

ax42.Box='on';

xlabel(ax42,'x');

ylabel(ax42,'e_n(x)');

title(ax42,sprintf('Right endpoint zoom, n =
%d',n_zoom),'FontWeight','bold','FontSize',12);

title(tl4,'Q1(c): Endpoint Error Zoom (Cubic
Spline)','FontSize',16,'FontWeight','bold');

ResultTable=table(n_list(:),(n_list(:)-
1),maxErr(:),rmsErr(:),xAtMax(:),'VariableNames',{'n_points','degree','max_abs_error',
'rms_error','x_at_max_error'});

disp('=====');

disp('Summary of Q1(c): cubic spline interpolation with evenly spaced points');

```



```

disp(ResultTable);

disp('=====');

function fpp=natural_spline_fpp(x,y)

n=length(x);

Delta=diff(x);

if n<=2

    fpp=zeros(n,1);

    return

end

A=zeros(n-2,n-2);

b=zeros(n-2,1);

for j=1:n-2

    if j>1

        A(j,j-1)=Delta(j)/6;

    end

    A(j,j)=(Delta(j)+Delta(j+1))/3;

    if j<n-2

        A(j,j+1)=Delta(j+1)/6;

    end

    b(j)=(y(j+2)-y(j+1))/Delta(j+1)-(y(j+1)-y(j))/Delta(j);

end

fpp=zeros(n,1);

fpp(2:n-1)=A\b;

end

function S4f=spline_eval(x,y,fpp,xq)

```

```

n=length(x);

s4f=zeros(size(xq));

for m=1:length(xq)

    xx=xq(m);

    if xx<=x(1)

        j=1;

    elseif xx>=x(n)

        j=n-1;

    else

        j=find(x<=xx,1,'last');

        if j==n

            j=n-1;

        end

    end

    Delta=x(j+1)-x(j);

    s4f(m)=((x(j+1)-xx)^3/(6*Delta)-Delta*(x(j+1)-xx)/6)*fpp(j)+((xx-
x(j))^3/(6*Delta)-Delta*(xx-x(j))/6)*fpp(j+1)+((y(j+1)-y(j))/Delta)*(xx-x(j))+y(j);

end

end

```

Q2(b)

Prompt:請依題目 notation 撰寫 Q2(b) 程式碼、將 DFT 定義正確轉為 FFT 可計算形式、檢查 fft 及 fftshift 與索引範圍設定。

```

clear; clc; close all;

a = 1.5;

N = 128;

x = -pi + (2*pi/N)*(0:N);

```

```

x_use = x(1:N);

g = exp(-a*x_use.^2);

F_raw = fft(g)/N;

F_shift = fftshift(F_raw);

j = -N/2:N/2-1;

gtilde_fft = exp(1i*pi*j).*F_shift;

gtilde_direct = zeros(size(j));

for n = 1:length(j)

    gtilde_direct(n) = (1/N)*sum(g.*exp(-1i*j(n)*x_use));

end

err_fft_direct = abs(gtilde_fft - gtilde_direct);

max_err = max(err_fft_direct);

max_imag = max(abs(imag(gtilde_fft)));

idx = abs(j) <= 8;

T =

table(j(idx).',real(gtilde_fft(idx)).',imag(gtilde_fft(idx)).',abs(gtilde_fft(idx)).',
abs(gtilde_direct(idx)).',err_fft_direct(idx).',...

'VariableNames',{'j','real_gtilde_fft','imag_gtilde_fft','abs_gtilde_fft','abs_gtilde_
direct','abs_error'});

disp(T)

fprintf('a = %.6f\n',a)

fprintf('N = %d\n',N)

fprintf('max |gtilde_fft - gtilde_direct| = %.6e\n',max_err)

fprintf('max imaginary part of gtilde_fft = %.6e\n',max_imag)

figure;

```

```

semilogy(j,abs(gtilde_fft),'o-','Linewidth',1.2,'MarkerSize',5);

grid on;

xlabel('$j$', 'Interpreter','latex');

ylabel('$|\tilde{g}_j(a,N)|$', 'Interpreter','latex');

title(sprintf('\mathrm{DFT} coefficients of g(x;a), a=%.3f, N=%d$',a,N), 'Interpreter','latex');

```

Q2(c)

Prompt: 請撰寫 Q2(c) 程式、設計可分別觀察 truncation、aliasing、leakage 的 (a,N)組合、檢查 Nyquist cutoff 與索引範圍。

```

clear; clc; close all;

Jmax = 256;

a1 = 20; N1 = 32;

a2 = 20; N2 = 16;

a3 = 0.1; N3 = 256;

r1 = case_data(a1,N1,Jmax);

r2 = case_data(a2,N2,Jmax);

r3 = case_data(a3,N3,Jmax);

fprintf('Case 1: truncation\n')

fprintf('a = %.6f, N = %d\n',r1.a,r1.N)

fprintf('relative band error = %.6e\n',r1.rel_band_err)

fprintf('max band error = %.6e\n',r1.max_band_err)

fprintf('tail energy ratio = %.6e\n\n',r1.tail_ratio)

fprintf('Case 2: aliasing\n')

fprintf('a = %.6f, N = %d\n',r2.a,r2.N)

fprintf('relative band error = %.6e\n',r2.rel_band_err)

fprintf('max band error = %.6e\n',r2.max_band_err)

```

```

fprintf('tail energy ratio      = %.6e\n\n',r2.tail_ratio)

fprintf('Case 3: leakage\n')

fprintf('a = %.6f, N = %d\n',r3.a,r3.N)

fprintf('relative band error    = %.6e\n',r3.rel_band_err)

fprintf('max band error          = %.6e\n',r3.max_band_err)

fprintf('tail energy ratio      = %.6e\n\n',r3.tail_ratio)

figure('Position',[100 80 900 950]);

tiledlayout(3,1,'TileSpacing','compact','Padding','compact');

nexttile

semilogy(r1.s_cont,r1.G_cont,'k-','Linewidth',1.2)

hold on

semilogy(r1.s_full,r1.fhat_full,'bo','MarkerSize',3)

semilogy(r1.s_band,abs(r1.gtilde),'rs','MarkerSize',4)

xline(r1.s_cut,'k--','Linewidth',1.0)

xline(-r1.s_cut,'k--','Linewidth',1.0)

grid on

xlabel('$$$', 'Interpreter','latex')

ylabel('magnitude', 'Interpreter','latex')

title(sprintf('Case 1: truncation, a = %.3f, N = %d',r1.a,r1.N), 'Interpreter','latex')

legend('$\frac{1}{2\pi}|G(s;a)|$', '$|\hat{f}_j(a)|$', '$|\tilde{g}_j(a,N)|$', 'Location'
, 'southwest', 'Interpreter','latex')

nexttile

semilogy(r2.s_cont,r2.G_cont,'k-','Linewidth',1.2)

hold on

semilogy(r2.s_full,r2.fhat_full,'bo','MarkerSize',3)

```

```

semilogy(r2.s_band,abs(r2.gtilde),'rs','MarkerSize',4)

xline(r2.s_cut,'k--','Linewidth',1.0)

xline(-r2.s_cut,'k--','Linewidth',1.0)

grid on

xlabel('$s$','Interpreter','latex')

ylabel('magnitude','Interpreter','latex')

title(sprintf('Case 2: aliasing, a = %.3f, N = %d',r2.a,r2.N),'Interpreter','latex')

legend('$\frac{1}{2\pi}|G(s;a)|$', '$|\hat{f}_j(a)|$', '$|\tilde{g}_j(a,N)|$', 'Location'
,'southwest','Interpreter','latex')

nexttile

semilogy(r3.s_cont,r3.G_cont,'k-','Linewidth',1.2)

hold on

semilogy(r3.s_full,r3.fhat_full,'bo','MarkerSize',3)

semilogy(r3.s_band,abs(r3.gtilde),'rs','MarkerSize',4)

xline(r3.s_cut,'k--','Linewidth',1.0)

xline(-r3.s_cut,'k--','Linewidth',1.0)

grid on

xlabel('$s$','Interpreter','latex')

ylabel('magnitude','Interpreter','latex')

title(sprintf('Case 3: leakage, a = %.3f, N = %d',r3.a,r3.N),'Interpreter','latex')

legend('$\frac{1}{2\pi}|G(s;a)|$', '$|\hat{f}_j(a)|$', '$|\tilde{g}_j(a,N)|$', 'Location'
,'southwest','Interpreter','latex')

exportgraphics(gcf,'q2c_all_cases.png','Resolution',300)

function r = case_data(a,N,Jmax)

j_full = -Jmax:Jmax;

```

```

s_full = j_full/(2*pi);

s_cont = linspace(-Jmax/(2*pi),Jmax/(2*pi),4000);

G_cont = (1/(2*pi))*sqrt(pi/a)*exp(-(pi^2)*(s_cont.^2)/a);

fhat_full = (1/(2*sqrt(pi*a)))*exp(-(j_full.^2)/(4*a));

[j_band,s_band,gtilde] = gtilde_fft(a,N);

fhat_band = (1/(2*sqrt(pi*a)))*exp(-(j_band.^2)/(4*a));

tail_ratio = sum(abs(fhat_full(abs(j_full)>N/2)).^2)/sum(abs(fhat_full).^2);

rel_band_err = norm(abs(gtilde)-abs(fhat_band))/norm(abs(fhat_band));

max_band_err = max(abs(abs(gtilde)-abs(fhat_band)));

r.a = a;

r.N = N;

r.j_full = j_full;

r.s_full = s_full;

r.s_cont = s_cont;

r.G_cont = G_cont;

r.fhat_full = fhat_full;

r.j_band = j_band;

r.s_band = s_band;

r.gtilde = gtilde;

r.fhat_band = fhat_band;

r.tail_ratio = tail_ratio;

r.rel_band_err = rel_band_err;

r.max_band_err = max_band_err;

r.s_cut = N/(4*pi);

end

```

```

function [j,s,gtilde] = gtilde_fft(a,N)

x = -pi + (2*pi/N)*(0:N-1);

g = exp(-a*x.^2);

F = fft(g)/N;

F = fftshift(F);

j = -N/2:N/2-1;

gtilde = exp(1i*pi*j).*F;

s = j/(2*pi);

end

```

Q2(d)

Prompt: 請依題目 notation 撰寫 Q2(d) 程式、實作直接 DFT 與 3/2 zero-padding de-aliasing 兩種方法、檢查 FFT 與索引流程是否正確。

```

clear; clc; close all;

a = 20;

N = 64;

M = 3*N/2;

Jmax = N/2-1;

kN = 0:N-1;

xN = -pi + 2*pi*kN/N;

gN = exp(-a*xN.^2);

hN = gN.^2;

jN = -N/2:N/2-1;

GstdN = fft(gN)/N;

gtilde_std_center = fftshift(GstdN);

gtilde_center = ((-1).^jN).*gtilde_std_center;

Hstd1 = fft(hN)/N;

```



```

htilde1_std_center = fftshift(Hstd1);

htilde1_center = ((-1).^jN).*htilde1_std_center;

fhat_exact_band = (1/(2*sqrt(pi*a))).*exp(-(jN.^2)/(4*a));

g_rel_band_err = norm(abs(gtilde_center)-
abs(fhat_exact_band))/norm(abs(fhat_exact_band));

g_max_band_err = max(abs(abs(gtilde_center)-abs(fhat_exact_band)));

gpad_std_center = zeros(1,M);

midM = M/2 + 1;

midN = N/2 + 1;

gpad_std_center(midM-N/2:midM+N/2-1) = gtilde_std_center;

Gpad_std_unshift = ifftshift(gpad_std_center)*M;

gM = ifft(Gpad_std_unshift);

hM = gM.^2;

Hpad_std_center = fftshift(fft(hM)/M);

htilde2_std_center = Hpad_std_center(midM-N/2:midM+N/2-1);

htilde2_center = ((-1).^jN).*htilde2_std_center;

diff_abs = abs(abs(htilde1_center)-abs(htilde2_center));

rel_nonlinear_err = norm(diff_abs)/norm(abs(htilde2_center));

max_nonlinear_err = max(diff_abs);

T =

table(jN.',real(htilde1_center).',real(htilde2_center).',abs(htilde1_center).',abs(hti
lde2_center).',diff_abs.','...

'VariableNames',{'j','real_htilde_1','real_htilde_2','abs_htilde_1','abs_htilde_2','ab
s_difference'});

disp(T(abs(jN)<=12,:))

```

```

fprintf('a = %.6f\n',a)

fprintf('N = %d\n',N)

fprintf('M = %d\n',M)

fprintf('g relative band error          = %.6e\n',g_rel_band_err)

fprintf('g max band error              = %.6e\n',g_max_band_err)

fprintf('relative nonlinear alias error = %.6e\n',rel_nonlinear_err)

fprintf('max nonlinear alias error      = %.6e\n',max_nonlinear_err)

figure('Position',[100 100 900 800]);

tiledlayout(2,1,'TileSpacing','compact','Padding','compact');

nexttile

semilogy(jN,abs(h_tilde1_center),'ro','MarkerSize',4)

hold on

semilogy(jN,abs(h_tilde2_center),'k-','Linewidth',1.2)

grid on

xlabel('$j$', 'Interpreter','latex')

ylabel('magnitude','Interpreter','latex')

title(sprintf('Comparison of $|\tilde{h}_{j,1}(a,N)|$ and $|\tilde{h}_{j,2}(a,N)|$,
$a= %.1f$, $N= %d$',a,N), 'Interpreter','latex')

legend('$|\tilde{h}_{j,1}(a,N)|$', '$|\tilde{h}_{j,2}(a,N)|$', 'Interpreter','latex','Lo
cation','best')

nexttile

semilogy(jN,diff_abs,'bo-','Linewidth',1.0,'MarkerSize',4)

grid on

xlabel('$j$', 'Interpreter','latex')

ylabel('absolute difference','Interpreter','latex')

```

```

title(sprintf('\bigl|\tilde{h}_{j,1}-\tilde{h}_{j,2}\bigr|$', $a=%.1f$,
$N=%d$',a,N),'Interpreter','latex')

exportgraphics(gcf,'q2d_compare.png','Resolution',300)

```

Q3(a)

Prompt:請撰寫 Q3(a) 的 MATLAB 程式、實作 forward difference 與 central difference、比較其誤差與收斂階數。

```

clear; clc; close all;

f = @(x) 2*sin(x) - 0.25*exp(x) - 1;

fp = @(x) 2*cos(x) - 0.25*exp(x);

Ns = [21 41 81 161 321 641];

num_cases = length(Ns);

h_list = zeros(num_cases,1);

err_forward = zeros(num_cases,1);

err_central = zeros(num_cases,1);

for n = 1:num_cases

    N = Ns(n);

    x = linspace(-1,1,N);

    h = x(2)-x(1);

    y = f(x);

    y_exact = fp(x);

    y_forward = zeros(size(x));

    y_forward(1:N-1) = (y(2:N)-y(1:N-1))/h;

    y_forward(N) = (y(N)-y(N-1))/h;

    y_central = zeros(size(x));

    y_central(2:N-1) = (y(3:N)-y(1:N-2))/(2*h);

    y_central(1) = (-3*y(1)+4*y(2)-y(3))/(2*h);

```

```

y_central(N) = (3*y(N)-4*y(N-1)+y(N-2))/(2*h);

h_list(n) = h;

err_forward(n) = max(abs(y_forward-y_exact));

err_central(n) = max(abs(y_central-y_exact));

if n == num_cases

    x_fine = x;

    y_exact_fine = y_exact;

    y_forward_fine = y_forward;

    y_central_fine = y_central;

end

end

order_forward = log(err_forward(1:end-1)./err_forward(2:end))./log(h_list(1:end-
1)./h_list(2:end));

order_central = log(err_central(1:end-1)./err_central(2:end))./log(h_list(1:end-
1)./h_list(2:end));

T1 =

table(Ns(:),h_list,err_forward,err_central,'VariableNames',{'N','h','max_error_forward
','max_error_central'});

disp(T1)

T2 =

table(Ns(2:end).',order_forward,order_central,'VariableNames',{'N_finer','observed_ord
er_forward','observed_order_central'});

disp(T2)

figure;

loglog(h_list,err_forward,'o-','LineWidth',1.2,'MarkerSize',6);

```

```

hold on;

loglog(h_list,err_central,'s-','Linewidth',1.2,'MarkerSize',6);

grid on;

xlabel('$h$', 'Interpreter','latex');

ylabel('$E_{\infty}$', 'Interpreter','latex');

title('Convergence of numerical differentiation schemes','Interpreter','latex');

legend('Forward difference','Central difference','Location','best');

exportgraphics(gcf,'q3a_convergence.png','Resolution',300);

fprintf('Finest grid N = %d\n',Ns(end))

fprintf('Forward max error = %.6e\n',err_forward(end))

fprintf('Central max error = %.6e\n',err_central(end))

```

Q3(b)

Prompt: 請 AI 撰寫 Q3(b) 的 MATLAB 程式、實作 2-point Gauss-Legendre quadrature 與 composite Simpson's 1/3 rule、比較其 numerical values 與 absolute errors °

```

clear; clc;

f = @(x) 2*sin(x) - 0.25*exp(x) - 1;

I_exact = -2 - (exp(1)-exp(-1))/4;

xg = [-1/sqrt(3), 1/sqrt(3)];

wg = [1, 1];

I_gauss2 = wg(1)*f(xg(1)) + wg(2)*f(xg(2));

err_gauss2 = abs(I_gauss2 - I_exact);

Ns = [3 5 9 17 33 65 129]';

num_cases = length(Ns);

h_list = zeros(num_cases,1);

I_simpson = zeros(num_cases,1);

```

```

err_simpson = zeros(num_cases,1);

for n = 1:num_cases

    N = Ns(n);

    x = linspace(-1,1,N);

    h = x(2)-x(1);

    y = f(x);

    I_simpson(n) = (h/3)*(y(1) + y(end) + 4*sum(y(2:2:end-1)) + 2*sum(y(3:2:end-2)));

    err_simpson(n) = abs(I_simpson(n) - I_exact);

    h_list(n) = h;

end

order_simpson = log(err_simpson(1:end-1)./err_simpson(2:end))./log(h_list(1:end-1)./h_list(2:end));

fprintf('Exact integral           = %.15f\n',I_exact)

fprintf('2-point Gauss-Legendre    = %.15f\n',I_gauss2)

fprintf('Gauss absolute error        = %.15e\n\n',err_gauss2)

T1 =

table(Ns,h_list,I_simpson,err_simpson,'VariableNames',{'N','h','I_simpson','abs_error_
simpson'});

disp(T1)

T2 =

table(Ns(2:end),order_simpson,'VariableNames',{'N_finer','observed_order_simpson'});

disp(T2)

fprintf('Finest Simpson N           = %d\n',Ns(end))

fprintf('Finest Simpson value       = %.15f\n',I_simpson(end))

fprintf('Finest Simpson error       = %.15e\n',err_simpson(end))

```

Q3(c)

Prompt: 請 AI 撰寫 Q3(c) 的 MATLAB 程式、實作 bisection method 與 Newton-Raphson method、在 $x=-5$ 附近自動搜尋最近兩個 sign-change intervals、比較兩種方法的 root values、iteration counts、residuals 與 absolute errors。

```
clear; clc;

f = @(x) 2*sin(x) - 0.25*exp(x) - 1;

fp = @(x) 2*cos(x) - 0.25*exp(x);

xscan = linspace(-10,0,400001);

yscan = f(xscan);

idx = find(yscan(1:end-1).*yscan(2:end) < 0);

a_all = xscan(idx);

b_all = xscan(idx+1);

mid_all = 0.5*(a_all+b_all);

[~,ord] = sort(abs(mid_all + 5));

a_sel = a_all(ord(1:2));

b_sel = b_all(ord(1:2));

a_sel = a_sel(:);

b_sel = b_sel(:);

tol = 1e-14;

maxit = 200;

root_ref = zeros(2,1);

root_bis = zeros(2,1);

root_newt = zeros(2,1);

iter_bis = zeros(2,1);

iter_newt = zeros(2,1);

res_bis = zeros(2,1);
```

```

res_newt = zeros(2,1);

err_bis = zeros(2,1);

err_newt = zeros(2,1);

order_bis = zeros(2,1);

order_newt = zeros(2,1);

for k = 1:2

    a = a_sel(k);

    b = b_sel(k);

    root_ref(k) = fzero(f,[a,b]);

    [rb,xb_hist] = my_bisection(f,a,b,tol,maxit);

    [rn,xn_hist] = my_newton(f,fp,0.5*(a+b),tol,maxit);

    root_bis(k) = rb;

    root_newt(k) = rn;

    iter_bis(k) = length(xb_hist)-1;

    iter_newt(k) = length(xn_hist)-1;

    res_bis(k) = abs(f(rb));

    res_newt(k) = abs(f(rn));

    err_bis(k) = abs(rb-root_ref(k));

    err_newt(k) = abs(rn-root_ref(k));

    eb = abs(xb_hist-root_ref(k));

    en = abs(xn_hist-root_ref(k));

    if length(eb) >= 4

        pb = log(eb(end)/eb(end-1))/log(eb(end-1)/eb(end-2));

        order_bis(k) = pb;

    else

```



```

        order_bis(k) = NaN;

    end

    if length(en) >= 4

        pn = log(en(end)/en(end-1))/log(en(end-1)/en(end-2));

        order_newt(k) = pn;

    else

        order_newt(k) = NaN;

    end

end

T =

table((1:2).',a_sel(:),b_sel(:),root_ref(:),root_bis(:),root_newt(:),iter_bis(:),iter_
newt(:),res_bis(:),res_newt(:),err_bis(:),err_newt(:),order_bis(:),order_newt(:),...
'VariableNames',{'root_id','a','b','reference_root','bisection_root','newton_root','it
er_bisection','iter_newton','res_bisection','res_newton','err_bisection','err_newton',
'observed_order_bisection','observed_order_newton'});

disp(T)

fprintf('Closest two sign-change intervals near x = -5:\n')

fprintf(' [%.15f, %.15f]\n',a_sel(1),b_sel(1))

fprintf(' [%.15f, %.15f]\n',a_sel(2),b_sel(2))

function [r,xhist] = my_bisection(f,a,b,tol,maxit)

fa = f(a);

fb = f(b);

xhist = [];

for n = 1:maxit

    c = 0.5*(a+b);

```

```

    fc = f(c);

    xhist = [xhist; c];

    if abs(fc) < tol || abs(b-a) < tol

        r = c;

        return

    end

    if fa*fc < 0

        b = c;

        fb = fc;

    else

        a = c;

        fa = fc;

    end

end

r = c;

end

function [r,xhist] = my_newton(f,fp,x0,tol,maxit)

xhist = x0;

x = x0;

for n = 1:maxit

    xnew = x - f(x)/fp(x);

    xhist = [xhist; xnew];

    if abs(f(xnew)) < tol || abs(xnew-x) < tol

        r = xnew;

        return

    end

```

end

x = xnew;

end

r = xnew;

end